Tài Liệu Kỹ Thuật: RAG Pipeline Visualizer

> **Dành cho ai?** Tài liệu này được viết cho người mới — không yêu cầu nền tảng IT chuyên sâu. Mỗi khái niệm kỹ thuật đều được giải thích bằng ngôn ngữ đơn giản, kèm ví dụ thực tế.

Mục Lục

1. Tổng quan về RAG
2. Giới thiệu Pipeline của Project
3. Stage 1 — Indexing (Lập chỉ mục)
    - 3.1 Bước 1: Loader — Đọc tài liệu PDF
    - 3.2 Bước 2: Chunking — Cắt đoạn
    - 3.3 Bước 3: Embedding — Chuyển thành vector
    - 3.4 Bước 4: Vector DB — Lưu trữ
4. Stage 2 — Generation (Sinh câu trả lời)
    - 4.1 Bước 7: Pre-retrieval — Xử lý câu hỏi
    - 4.2 Bước 8: Retrieval — Tìm kiếm
    - 4.3 Bước 9: Post-retrieval — Lọc & xếp hạng
    - 4.4 Bước 10: Prompt — Xây dựng câu hỏi cho LLM
    - 4.5 Bước 11: Generation — Sinh câu trả lời
5. Luồng hoạt động toàn bộ Pipeline
6. Scaling & Guardrails — Mở rộng ứng dụng
7. So sánh cấu hình và lời khuyên thực tế
8. Glossary — Bảng thuật ngữ

---

1. Tổng quan về RAG

1.1 Vấn đề mà RAG giải quyết

Hãy hình dung bạn thuê một chuyên gia tư vấn luật rất giỏi — người này đã đọc hàng triệu trang sách luật từ trước đến nay. Nhưng có một vấn đề: họ bị "đóng băng" tại thời điểm tốt nghiệp. Họ không biết nghị định nào vừa được ban hành tuần trước, không có trong tay hồ sơ nội bộ của công ty bạn, và đôi khi tự tin trả lời sai vì nhớ nhầm.

Đây chính xác là giới hạn của các mô hình AI ngôn ngữ lớn (**LLM** — Large Language Model) như GPT-4 hay Claude:

Vấn đề	Biểu hiện	Ví dụ
Tri thức lỗi thời	Trả lời dựa trên thông tin cũ	Không biết sản phẩm mới ra mắt tháng trước
Không biết tài liệu nội bộ	Bịa ra câu trả lời hoặc từ chối	Hỏi về quy trình nội bộ công ty → LLM không biết
Hallucination (Ảo giác)	Tự tin đưa ra thông tin sai	Trích dẫn một điều luật không tồn tại

RAG (Retrieval-Augmented Generation — Sinh văn bản tăng cường bằng truy xuất) ra đời để giải quyết chính xác 3 vấn đề này. Thay vì để LLM "tự nhớ", RAG cho phép LLM **tra cứu tài liệu thực** trước khi trả lời — giống như một chuyên gia được phép mở sách tham khảo trước khi đưa ra ý kiến.

1.2 RAG hoạt động như thế nào?

Ý tưởng cốt lõi rất đơn giản:

Người dùng hỏi → Hệ thống tìm đoạn tài liệu liên quan → LLM đọc tài liệu đó → Trả lời

Ví dụ thực tế:

Câu hỏi: "Chính sách hoàn tiền của công ty chúng tôi là bao nhiêu ngày?" **Không có RAG:** LLM đoán "30 ngày" hoặc "14 ngày" dựa trên chính sách phổ biến nó đã đọc — có thể sai. **Có RAG:** Hệ thống tìm đúng đoạn trong tài liệu nội bộ: *"Khách hàng có quyền yêu cầu hoàn tiền trong vòng 45 ngày..."* → LLM trả lời chính xác **45 ngày**, kèm trích dẫn nguồn.

1.3 RAG so với các phương án khác

Ngoài RAG, có hai cách khác để "mở rộng tri thức" cho LLM:

Tiêu chí	RAG	Fine-tuning	Long-context LLM
Cập nhật tri thức mới	✅ Ngay lập tức	❌ Phải train lại	⚠️ Phải nhét hết vào
Chi phí	Trung bình	Cao (cần GPU)	Rất cao (token đắt)
Trích dẫn nguồn	✅ Biết chính xác	❌ Không rõ	⚠️ Không chắc
Kho dữ liệu lớn	✅ Tốt	❌ Không thực tế	❌ Không thể
Giảm hallucination	✅ Đáng kể	⚠️ Vẫn có	⚠️ Lost-in-middle

> 💡 **Thực tế 2025:** Hầu hết hệ thống production kết hợp cả ba — RAG cho tri thức thay đổi, fine-tuning cho phong cách/domain đặc thù, long-context cho phân tích sâu tài liệu nhỏ.

1.4 Lịch sử tiến hóa RAG

RAG không xuất hiện một sớm một chiều mà trải qua 5 giai đoạn phát triển:

Năm	Giai đoạn	Đặc điểm
2020	Khai sinh (Lewis et al.)	Bài báo gốc — kết hợp DPR + BART, train end-to-end
2021–2022	Naive RAG	Pipeline đơn giản với LLM có sẵn, dễ xây nhưng nhiều lỗi
2023	Advanced RAG	Nhiều kỹ thuật tinh vi: reranking, hybrid search, query transform
2024	Modular RAG	Lắp ghép linh hoạt như LEGO — mỗi bước là module độc lập
2025–2026	Agentic RAG	AI tự quyết định chiến lược, tự lên kế hoạch retrieve

- **Naive RAG (2021-2022):** Chỉ cần tìm kiếm vector + LLM. Đơn giản nhưng nhiều lỗi.
- **Advanced RAG (2023):** Thêm reranking, hybrid search, query transformation. Chất lượng tốt hơn nhiều.
- **Modular RAG (2023-2024):** Mỗi bước là một module độc lập, có thể hoán đổi như LEGO.
- **Agentic RAG (2024-2026):** LLM tự lên kế hoạch, tự quyết định cần retrieve gì, từ đâu, bao nhiêu lần.

RAG-pipeline-visualizer của project này thuộc thế hệ **Modular RAG** — mỗi bước đều có thể cấu hình độc lập.

---

2. Giới thiệu Pipeline của Project

2.1 Tổng quan kiến trúc

Pipeline gồm **2 stage** hoàn toàn độc lập:

Stage 1 — INDEXING (Chuẩn bị, chạy offline)

`[PDF]` → `[Loader]` → `[Chunking]` → `[Embedding]` → `[Vector DB]`

Stage 2 — GENERATION (Hỏi đáp, chạy mỗi query)

`[Query]` → `[Pre-retrieval]` → `[Retrieval]` → `[Post-retrieval]` → `[Prompt]` → `[Generation]` → `[Câu trả lời]`

Stage 1 (Indexing) chỉ chạy **một lần** khi bạn upload tài liệu mới. Kết quả được lưu vào Vector Database để dùng mãi.

Stage 2 (Generation) chạy **mỗi khi** người dùng đặt câu hỏi. Tốc độ phải nhanh (thường dưới 5 giây).

2.2 Tại sao có đánh số bước từ 1 đến 4 rồi nhảy lên 7?

Đây là quy ước của Modular RAG — các bước 5 và 6 được "dành chỗ" cho các kỹ thuật nâng cao hơn (như evaluation feedback loop, memory module) nhưng chưa được implement trong phiên bản hiện tại. Bước đánh số không ảnh hưởng đến cách pipeline hoạt động.

2.3 Điểm đặc biệt của project

Project này không chỉ chạy RAG — nó **trực quan hóa** từng bước để bạn thấy rõ:

- Tài liệu được parse ra như thế nào
- Các chunk trông như thế nào sau khi cắt
- Vector embedding nhìn ra sao (similarity heatmap)
- Chunk nào được retrieve, score bao nhiêu
- Prompt thực tế gửi lên LLM là gì
- Token usage và thời gian mỗi bước

Đây là công cụ học tập và thử nghiệm lý tưởng — bạn có thể đổi một cấu hình và quan sát ngay kết quả thay đổi như thế nào.

---

3. Stage 1 — Indexing

> **Mục tiêu của Stage 1:** Chuyển đổi tài liệu PDF thô thành một "thư viện số" có thể tìm kiếm theo ngữ nghĩa trong mili-giây.

3.1 Bước 1: Loader

Loader là bước đầu tiên — đọc file PDF và chuyển nội dung thành dạng text mà máy tính có thể xử lý.

Nghe có vẻ đơn giản, nhưng đây thực ra là bước **quan trọng nhất và khó nhất** trong toàn bộ pipeline. Lý do: PDF là định dạng được thiết kế để **in đẹp**, không phải để **đọc máy**. Một file PDF có thể chứa:

- Text thông thường (dễ đọc)
- Bảng biểu với cấu trúc phức tạp
- Hình ảnh, biểu đồ, sơ đồ
- Công thức toán học
- Text được nhúng trong ảnh (PDF scan)
- Layout nhiều cột

Nếu Loader đọc sai → toàn bộ pipeline phía sau sẽ hoạt động trên dữ liệu rác → câu trả lời sai dù dùng LLM tốt nhất.

3.1.1 PyPDF

Giới thiệu: Thư viện Python thuần túy, không cần cài thêm phần mềm bên ngoài. Trích xuất text trực tiếp từ text layer của PDF.

Cơ chế hoạt động: PDF file → Đọc từng trang → Trích xuất text layer → Ghép lại thành string

Ưu điểm:

- Cực kỳ nhanh (vài giây cho file 100 trang)
- Không cần cấu hình gì thêm
- Hoạt động offline hoàn toàn
- Tiêu thụ ít RAM

Nhược điểm:

- Không xử lý được bảng — nội dung bảng bị đọc thành chuỗi text lộn xộn
- Không nhận diện công thức toán học
- PDF scan (ảnh) → trả về rỗng hoàn toàn
- PDF nhiều cột thường bị đọc nhầm thứ tự dòng

Khi nào nên dùng:

- PDF văn bản thuần (hợp đồng, báo cáo dạng prose, tài liệu text)
- Prototype nhanh, không cần độ chính xác cao
- Khi tốc độ là ưu tiên số 1

Ví dụ thực tế: Đọc một file biên bản họp 20 trang — PyPDF hoàn thành trong 0.5 giây và cho kết quả tốt nếu file không có bảng hay hình ảnh.

3.1.2 PyMuPDF (fitz)

Giới thiệu: Binding Python của thư viện MuPDF — một engine render PDF chuyên nghiệp. Chất lượng cao hơn PyPDF đáng kể, đặc biệt với layout phức tạp.

Cơ chế hoạt động: PDF → MuPDF engine render → Trích xuất text với tọa độ → Sắp xếp theo vị trí → Output PyMuPDF "nhìn" PDF theo tọa độ (x, y) của từng ký tự, nên có thể tái tạo thứ tự đọc chính xác hơn.

Ưu điểm:

- Nhanh hơn pdfplumber, chậm hơn PyPDF một chút
- Xử lý layout phức tạp (nhiều cột) tốt hơn PyPDF
- Trích xuất metadata phong phú (font, màu sắc, tọa độ)
- Có thể render từng trang thành ảnh (hữu ích cho VLM)
- Hỗ trợ PDF mã hóa

Nhược điểm:

- Bảng phức tạp vẫn không được xử lý tốt
- Công thức toán học không được nhận diện
- Không xử lý PDF scan

Khi nào nên dùng:

- PDF layout đơn giản đến trung bình
- Cần metadata chi tiết (vị trí text, font)
- Khi cần tốc độ nhanh nhưng chất lượng tốt hơn PyPDF

3.1.3 PDFPlumber

Giới thiệu: Được xây dựng trên pdfminer, PDFPlumber chuyên sâu về **trích xuất bảng** từ PDF. Đây là lựa chọn tốt nhất cho PDF nhiều bảng có text layer.

Cơ chế hoạt động: PDF → Phân tích đường kẻ + căn chỉnh text → Nhận diện cấu trúc bảng → Xuất Markdown table PDFPlumber phân tích các đường kẻ ngang/dọc và căn chỉnh của text để "đoán" ra cấu trúc bảng.

Ưu điểm:

- **Trích xuất bảng tốt nhất** trong số các loader không dùng AI
- Giữ nguyên cấu trúc bảng dưới dạng Markdown
- Trích xuất tọa độ chính xác của từng element
- Xử lý PDF financial reports, spreadsheet-heavy docs rất tốt

Nhược điểm:

- Chậm hơn PyPDF và PyMuPDF (~3-5x)
- Bảng không có đường kẻ rõ ràng (borderless table) vẫn khó xử lý
- PDF scan vẫn không xử lý được
- Tốn RAM hơn với file lớn

Khi nào nên dùng:

- Báo cáo tài chính, bảng số liệu
- Tài liệu hành chính có nhiều bảng biểu
- Hợp đồng với điều khoản dạng bảng

Ví dụ thực tế: Một báo cáo tài chính quý có 50 bảng số liệu — PDFPlumber có thể trích xuất ra đúng cấu trúc bảng Markdown, trong khi PyPDF sẽ cho ra một đống số lộn xộn.

3.1.4 Marker ⭐ (Khuyến nghị cho PDF phức tạp)

Giới thiệu: Marker là công cụ convert PDF → Markdown sử dụng AI (deep learning). Được phát triển bởi VikParuchuri, đây là loader chất lượng cao nhất cho PDF phức tạp có bảng, công thức, và hình ảnh.

Cơ chế hoạt động: PDF → Phân tích layout bằng vision model

- Nhận diện từng element (text, bảng, hình, công thức)
- Trích xuất text từng vùng
- Convert bảng → Markdown table
- Convert công thức → LaTeX
- Mô tả hình ảnh (nếu bật)
- Ghép thành Markdown có cấu trúc

Marker không chỉ đọc text — nó **hiểu cấu trúc** của tài liệu và tái tạo nó dưới dạng Markdown.

Ưu điểm:

- Chất lượng output Markdown xuất sắc, giữ nguyên heading hierarchy
- Bảng được convert sang Markdown table chính xác
- Công thức toán học → LaTeX (rất quan trọng với tài liệu khoa học/kỹ thuật)
- Xử lý PDF multi-column layout tốt
- Output Markdown kết hợp cực kỳ tốt với `format_aware` chunking

Nhược điểm:

- Chậm hơn nhiều so với các loader khác (~10-30x)
- Tải model AI ~1.5 GB trong lần chạy đầu tiên
- Cần RAM ít nhất 4 GB, GPU tăng tốc đáng kể
- Cần kết nối internet lần đầu

Khi nào nên dùng:

- Tài liệu học thuật (research paper, luận văn)
- Tài liệu kỹ thuật có công thức (toán, vật lý, kỹ thuật)
- PDF có bảng phức tạp và hình ảnh quan trọng
- Khi chất lượng quan trọng hơn tốc độ

Ví dụ thực tế: Một paper nghiên cứu AI 20 trang với 10 bảng kết quả thực nghiệm và 15 công thức toán — Marker sẽ tạo ra Markdown đẹp, các công thức dưới dạng `$E = mc^2$`, bảng đúng cấu trúc. PyPDF sẽ cho ra một đống ký tự lộn xộn.

3.1.5 Docling

Giới thiệu: Được phát triển bởi IBM Research, Docling là parser tài liệu AI-powered với kiến trúc riêng biệt. Hỗ trợ nhiều định dạng hơn Marker (PDF, Word, PowerPoint, Excel, HTML).

Cơ chế hoạt động: Document → DoclingDocument model (phân tích layout)

- TableFormer model (nhận diện bảng)
- OCR engine (nếu cần)
- Export sang Markdown / JSON / DocChunks

Ưu điểm:

- Hỗ trợ nhiều định dạng file (không chỉ PDF)
- TableFormer model chuyên biệt cho bảng — rất chính xác
- Output JSON có cấu trúc phong phú (bounding box, confidence score)
- Tích hợp tốt với LlamaIndex và LangChain
- Chất lượng Markdown tương đương Marker

Nhược điểm:

- Thời gian xử lý lâu (tương tự Marker)
- Tải nhiều model hơn (~2-3 GB)
- Cần cấu hình phức tạp hơn cho use case nâng cao

Khi nào nên dùng:

- Corpus đa dạng định dạng (không chỉ PDF)
- Cần metadata phong phú về cấu trúc tài liệu
- Dự án dùng LlamaIndex ecosystem

3.1.6 Unstructured

Giới thiệu: ETL (Extract, Transform, Load) platform cho tài liệu phi cấu trúc. Đây là giải pháp "tất cả trong một" với khả năng OCR mạnh nhất.

Cơ chế hoạt động: Document → Phát hiện loại file → Chọn parser phù hợp

- OCR (nếu là ảnh/scan)
- Phân loại element (Title, NarrativeText, Table, Image, ListItem...)
- Export thành list[Element]

Ưu điểm:

- **OCR tốt nhất** cho PDF scan (kết hợp Tesseract/PaddleOCR)
- Tự động phát hiện loại tài liệu
- Phân loại element chi tiết (biết đâu là tiêu đề, đâu là bảng, đâu là đoạn văn)
- Hỗ trợ 20+ định dạng file
- Có cloud API (Unstructured.io) cho scale lớn

Nhược điểm:

- Chậm nhất trong tất cả loader
- Cần cài Tesseract/Poppler riêng (phức tạp trên Windows)
- Chi phí nếu dùng cloud API
- Output cần post-processing nhiều hơn

Khi nào nên dùng:

- PDF scan (ảnh chụp tài liệu, không có text layer)
- Kho tài liệu hỗn hợp nhiều định dạng
- Cần phân loại element chi tiết cho downstream processing

3.1.7 Tổng kết: Chọn Loader nào?

Tài liệu của bạn là gì?

- **PDF scan / ảnh chụp?** → Unstructured (+ OCR)
- **PDF học thuật / công thức?** → Marker ⭐
- **PDF nhiều bảng tài chính?** → PDFPlumber
- **PDF văn bản thông thường?** → PyMuPDF hoặc PyPDF
- **Nhiều định dạng khác nhau?** → Docling

OCR Engines (dùng kèm với Unstructured hoặc Marker):

Engine	Điểm mạnh	Khi dùng
`tesseract`	Nhanh, 100+ ngôn ngữ	Văn bản in rõ trên nền trắng
`paddleocr`	Tiếng Việt, CJK, layout phức tạp	PDF tiếng Việt scan
`easyocr`	Đa ngôn ngữ, CPU-friendly	Khi không có GPU
`surya`	Chất lượng cao nhất, transformer-based	Chất lượng > tốc độ

3.2 Bước 2: Chunking

Chunking (phân đoạn) là bước cắt tài liệu dài thành các đoạn nhỏ hơn. Đây là **bài toán khó nhất** trong RAG — không có đáp án hoàn hảo cho mọi trường hợp.

Tại sao cần Chunking?

Có 2 lý do:

1. **Embedding model** có giới hạn độ dài đầu vào (thường 512-8192 token). Tài liệu 100 trang không thể nhét vào một lần.
2. **Precision của retrieval:** Nếu bạn embed cả quyển sách thành 1 vector, khi tìm kiếm sẽ không chính xác. Chunk nhỏ hơn → vector đặc trưng hơn → tìm kiếm chính xác hơn.

Bài toán cân bằng (Chunking Dilemma):

> Chunk quá nhỏ (< 100 ký tự):
> ✅ Precision cao — vector đặc trưng, ít nhiễu
> ❌ Thiếu context — câu trả lời cụt, không đủ nghĩa
> Chunk quá lớn (> 2000 ký tự):
> ✅ Context đầy đủ — LLM có đủ thông tin

> ❌ Precision thấp — embedding bị "pha loãng"
> ❌ Tốn token LLM — đắt và chậm
> Ngưỡng thực tế tốt: 500-1500 ký tự (tùy loại tài liệu)

3.2.1 Recursive Chunking ⭐

Giới thiệu: Chiến lược mặc định và phổ biến nhất. Cắt tài liệu theo danh sách separator ưu tiên, từ lớn đến nhỏ — đoạn văn trước, rồi dòng, rồi câu, cuối cùng mới cắt theo ký tự.

Cơ chế hoạt động: Thử cắt theo: ["\n\n", "\n", ". ", " ", ""] (đoạn) (dòng) (câu) (từ) (ký tự)

- Nếu chunk còn quá lớn, dùng separator nhỏ hơn
- Dừng khi chunk đạt kích thước mong muốn

Ví dụ với chunk_size = 500:
Đoạn 1: 800 ký tự → Cắt theo "\n" → Chunk A (480 ký tự) + Chunk B (320 ký tự) ✅
Đoạn 2: 300 ký tự → Nhỏ hơn 500 → Giữ nguyên ✅

Ưu điểm:

- **Tốt nhất cho hầu hết trường hợp** — đây là lý do nó là mặc định
- Tôn trọng ranh giới tự nhiên của văn bản (ưu tiên cắt ở đoạn)
- Không bao giờ cắt giữa chừng một từ
- Overlap giúp tránh mất thông tin ở ranh giới

Nhược điểm:

- Không "hiểu" ngữ nghĩa — có thể cắt giữa chừng một ý quan trọng
- Separator list phải được điều chỉnh cho từng ngôn ngữ/loại tài liệu

Tham số quan trọng:

- `chunk_size`: Số ký tự tối đa mỗi chunk. Khuyến nghị **512–1500**.

- `chunk_overlap`: Số ký tự chồng lấp giữa 2 chunk liên tiếp. Khuyến nghị **10-15% của chunk_size**.

Chunk overlap là gì? Hãy tưởng tượng bạn đọc một cuốn sách và đánh dấu từng đoạn. Overlap là việc đoạn sau bắt đầu từ một vài câu trước khi đoạn trước kết thúc. Điều này đảm bảo thông tin nằm ở ranh giới 2 chunk không bị mất.

Khi nào nên dùng: Luôn là lựa chọn đầu tiên cần thử. Phù hợp với mọi loại tài liệu văn bản.

3.2.2 Token-based Chunking

Giới thiệu: Thay vì đếm ký tự, đếm **token** (đơn vị ngôn ngữ mà LLM xử lý). Dùng thư viện Tiktoken của OpenAI để đếm chính xác.

Token là gì? Một token thường là ~4 ký tự tiếng Anh, hoặc 1-2 ký tự tiếng Việt/CJK. Ví dụ: "Hello world" = 2 token, nhưng "Xin chào thế giới" ≈ 8-10 token vì tiếng Việt cần nhiều token hơn.

Tại sao quan trọng với tiếng Việt?

LLM tính phí và giới hạn theo **token**, không phải ký tự. Nếu bạn set chunk_size = 1000 ký tự nhưng tài liệu tiếng Việt, thực tế có thể là 2000+ token — vượt giới hạn embedding model mà không biết.

Token-based chunking đảm bảo mỗi chunk **không bao giờ vượt quá giới hạn token** của embedding model.

Ưu điểm:

- Kiểm soát chính xác số token — không bị silent truncation
- Phù hợp với tài liệu tiếng Việt, CJK (1 ký tự = nhiều token)
- Tránh lãng phí context window của LLM

Nhược điểm:

- Chậm hơn một chút (cần tokenize)
- Khó hình dung "chunk_size = 500 token trông như thế nào"
- Mỗi model dùng tokenizer khác nhau

Khi nào nên dùng:

- Corpus tiếng Việt hoặc ngôn ngữ CJK
- Khi dùng embedding model có giới hạn token chặt chẽ
- Production system cần kiểm soát chi phí API chính xác

3.2.3 Format-aware Chunking ⭐

Giới thiệu: Nhận diện cấu trúc định dạng của tài liệu (Markdown heading, code block, HTML tag) và cắt theo ranh giới đó thay vì theo ký tự.

Cơ chế hoạt động: Input: Markdown từ Marker/Docling "# Chương 1: Giới thiệu\n\n## 1.1 Tổng quan\n..."

- Nhận diện: # = H1, ## = H2, --- = separator
- Cắt theo hierarchy: mỗi H2 section = 1 chunk
- Giữ nguyên code blocks (không cắt giữa chừng)
- Giữ nguyên bảng Markdown

Ưu điểm:

- Mỗi chunk là **một đơn vị có nghĩa** (một section, một đoạn hoàn chỉnh)
- Không bao giờ cắt giữa bảng hay code block

- Kết hợp xuất sắc với Marker loader (Markdown đầu vào → format-aware cắt đúng section)
- Chunk giữ nguyên heading → LLM biết context rõ hơn

Nhược điểm:

- Phụ thuộc vào chất lượng output của Loader (nếu Loader cho ra plain text → không hiệu quả)
- Chunk size không đều (section ngắn vs section dài)
- Cần cấu hình `format_type` (markdown/html/code)

Khi nào nên dùng:

- **Bắt buộc dùng khi Loader là Marker hoặc Docling** (output là Markdown)
- Tài liệu có cấu trúc heading rõ ràng
- Code documentation, technical specs
- Kết hợp lý tưởng: Marker loader + Format-aware chunking

Ví dụ thực tế — tài liệu được cắt theo heading:

> **# Chính sách hoàn tiền** ← Chunk 1 bắt đầu từ đây
> **## 1. Điều kiện áp dụng** Khách hàng có thể... ← Chunk 1 kết thúc khi gặp ## tiếp theo
> **## 2. Quy trình hoàn tiền** ← Chunk 2 bắt đầu Để yêu cầu hoàn tiền... ← Chunk 2

Thay vì cắt ngẫu nhiên giữa chừng một điều khoản!

3.2.4 Sentence-aware Chunking

Giới thiệu: Dùng NLTK sentence tokenizer để xác định ranh giới câu, đảm bảo chunk luôn kết thúc ở cuối một câu hoàn chỉnh.

Cơ chế hoạt động: "Câu 1 dài... Câu 2. Câu 3 cũng dài."

- NLTK tách: ["Câu 1 dài...", "Câu 2.", "Câu 3 cũng dài."]
- Gom các câu lại cho đến khi gần đạt chunk_size
- Không bao giờ cắt giữa câu

Ưu điểm:

- Mỗi chunk là **câu hoàn chỉnh** — không bị cụt
- Đặc biệt tốt cho Q&A ngắn, FAQ
- Chunk có nghĩa tự nhiên hơn

Nhược điểm:

- Phụ thuộc vào NLTK sentence model — tiếng Việt cần model riêng
- Chunk size không đều (câu ngắn vs câu dài)
- Câu rất dài (điều khoản hợp đồng) vẫn là 1 chunk lớn

Khi nào nên dùng:

- Tài liệu FAQ, câu hỏi — câu trả lời
- Văn học, báo chí
- Khi câu hoàn chỉnh là đơn vị ý nghĩa quan trọng

3.2.5 Semantic Chunking

Giới thiệu: Cắt tài liệu dựa trên **sự thay đổi chủ đề** thay vì kích thước cố định. Dùng embedding để phát hiện khi nào nội dung chuyển sang topic mới.

Cơ chế hoạt động: Tài liệu → Chia thành câu nhỏ

- Embed từng câu
- Tính cosine similarity giữa câu liền kề
- Khi similarity đột ngột giảm → đây là ranh giới topic mới
- Gom các câu cùng topic lại thành 1 chunk

Hình dung: Giống như bạn đọc một bài báo và nhận ra "Ồ, đoạn này đang chuyển sang nói về chủ đề khác rồi" — Semantic chunking tự động làm điều đó.

Ưu điểm:

- Mỗi chunk là **một ý trọn vẹn** — không cắt giữa chừng một luận điểm
- Chunk size linh hoạt theo nội dung thực tế
- Hiệu quả nhất về mặt ngữ nghĩa

Nhược điểm:

- Chậm nhất — phải embed từng câu, sau đó mới chunk
- Tốn API call embedding (nếu dùng OpenAI)
- Chunk size không kiểm soát được chính xác
- Kết quả phụ thuộc vào threshold similarity (cần tune)

Khi nào nên dùng:

- Văn bản đa chủ đề (white paper, sách giáo khoa)
- Khi chất lượng semantic là ưu tiên, không quan tâm chi phí
- Corpus quan trọng cần đầu tư thời gian xử lý

3.2.6 Hierarchical Chunking (Parent-Child)

Giới thiệu: Tạo ra 2 cấp chunk — **Parent** (lớn, chứa đủ context) và **Child** (nhỏ, dùng để tìm kiếm chính xác). Khi retrieve, tìm bằng Child nhưng trả về Parent cho LLM.

Cơ chế hoạt động: Tài liệu → Parent chunk (1500 ký tự) — lưu vào document store

- Mỗi Parent cắt thành 3-4 Child chunk (400 ký tự) — lưu vào vector DB Khi query:
- Tìm kiếm trong vector DB (Child chunks — nhỏ → chính xác)
- Lấy ID của Parent tương ứng
- Trả Parent về cho LLM (lớn → đủ context)

Ví dụ thực tế:

Parent: "Điều 5. Quyền và nghĩa vụ của bên A... [toàn bộ điều khoản 1500 ký tự]"
Child 1: "Điều 5.1 Bên A có quyền yêu cầu..." ← Vector DB tìm thấy cái này
Child 2: "Điều 5.2 Bên A có nghĩa vụ nộp..."
Child 3: "Điều 5.3 Trong trường hợp vi phạm..."

- Query: "Bên A phải nộp tiền khi nào?"
- Retrieval: Tìm Child 2 (khớp nhất)
- Trả về: Parent đầy đủ (vì Child 2 thuộc về Parent này)
- LLM đọc toàn bộ Điều 5 → hiểu đầy đủ ngữ cảnh

Ưu điểm:

- **Kết hợp tốt nhất của 2 thế giới:** Precision của chunk nhỏ + Context của chunk lớn
- Giải quyết triệt để bài toán chunk quá nhỏ thiếu context
- Đặc biệt hiệu quả cho tài liệu pháp lý, hợp đồng

Nhược điểm:

- Lưu trữ gấp đôi (cả Parent và Child)
- Phức tạp hơn trong implementation

- Cần vector DB hỗ trợ parent-child relationship

Khi nào nên dùng:

- Corpus lớn, tài liệu pháp lý, hợp đồng
- Khi câu trả lời đòi hỏi đọc cả một section dài
- Production system cần chất lượng cao

3.2.7 Contextual Chunking

Giới thiệu: Kỹ thuật nâng cao từ Anthropic (2024) — dùng LLM để **thêm context** vào mỗi chunk trước khi embed. Mỗi chunk được thêm một đoạn mô tả ngắn về vị trí của nó trong tài liệu.

Cơ chế hoạt động: Chunk gốc: "Điều khoản này áp dụng trong 30 ngày." LLM thêm context: "[Từ tài liệu Chính sách hoàn tiền, Điều 3.2 về thời hạn yêu cầu] Điều khoản này áp dụng trong 30 ngày."

- Embed chunk đã thêm context

Tại sao hiệu quả? Chunk gốc "áp dụng trong 30 ngày" rất mơ hồ khi đứng một mình. Nhưng khi đã có context "chính sách hoàn tiền, điều 3.2", vector embedding sẽ chính xác hơn nhiều.

Ưu điểm:

- Cải thiện recall đáng kể (theo Anthropic: +49%)
- Chunk "hiểu chính nó" hơn — ngữ nghĩa phong phú hơn
- Giải quyết vấn đề chunk mất context khi tách ra khỏi tài liệu gốc

Nhược điểm:

- Tốn nhiều API call LLM (mỗi chunk cần 1 call)
- Chi phí cao với corpus lớn
- Thời gian indexing lâu hơn nhiều

Khi nào nên dùng:

- Production system, có budget LLM
- Corpus quan trọng, cần recall cao nhất có thể
- Tài liệu dày đặc, nhiều tham chiếu chéo

3.2.8 Deduplication (Loại bỏ trùng lặp)

Giới thiệu: Sau khi chunking, tự động phát hiện và loại bỏ các chunk gần giống nhau bằng **MinHash** — thuật toán hashing hiệu quả cho phép so sánh nhanh hàng nghìn chunk.

Tại sao cần? Tài liệu thực tế thường có nội dung trùng lặp: headers/footers xuất hiện ở mọi trang, chính sách được nhắc lại nhiều lần, template text. Nếu không dedup, Vector DB sẽ chứa nhiều chunk gần giống nhau → khi retrieve sẽ trả về nhiều bản copy → lãng phí context window.

Ưu điểm:

- Giảm kích thước Vector DB
- Cải thiện diversity của retrieved context
- Giảm chi phí embedding

Khi nào nên dùng: Luôn nên bật khi corpus có nhiều file có nội dung trùng nhau (template documents, hợp đồng mẫu, báo cáo định kỳ).

3.2.9 Tổng kết: Chọn Chunking nào?

Chunking	Tốc độ	Chất lượng	Dùng khi
`recursive` ⭐	⚡⚡⚡	★★★	Mặc định — thử đầu tiên
`token_based`	⚡⚡	★★★	Tiếng Việt, kiểm soát token
`format_aware`	⚡⚡⚡	★★★★	Output từ Marker/Docling
`sentence_aware`	⚡⚡	★★★	FAQ, văn bản ngắn
`semantic`	⚡	★★★★★	Văn bản đa chủ đề, chất lượng cao
`hierarchical`	⚡⚡	★★★★★	Production, tài liệu pháp lý
`contextual`	⚡ (chậm)	★★★★★	Production, budget LLM

Cặp kết hợp khuyến nghị:

- `marker` + `format_aware` — PDF phức tạp
- `pypdf` + `recursive` — Prototype nhanh
- `marker` + `hierarchical` — Production tốt nhất

3.3 Bước 3: Embedding

Embedding là bước chuyển đổi từng chunk text thành một **vector số** — danh sách hàng trăm đến hàng nghìn con số đại diện cho "ý nghĩa" của đoạn text đó.

Hình dung bằng ví dụ thực tế:

Hãy tưởng tượng bạn vẽ một bản đồ ý nghĩa trong không gian 3 chiều (thực tế là 1536 chiều):
"Con mèo ngồi trên ghế" → vector [0.2, 0.8, 0.1, ...]
"Chú mèo đang ngồi trên bàn" → vector [0.21, 0.79, 0.12, ...] ← rất gần nhau!
"Xe ô tô màu đỏ" → vector [0.9, 0.1, 0.7, ...] ← xa nhau

- Cosine similarity("mèo ngồi ghế", "mèo ngồi bàn") = 0.97 (rất giống)
- Cosine similarity("mèo ngồi ghế", "xe ô tô") = 0.12 (không liên quan)

Đây là cơ chế cho phép tìm kiếm "ngữ nghĩa" — không cần khớp từ khóa chính xác.

3.3.1 Dense Embedding (Vector dày)

Dense embedding là loại embedding **phổ biến nhất** — chuyển text thành vector có tất cả chiều đều khác 0.

OpenAI — text-embedding-3-small ⭐

Embedding model mặc định của OpenAI. "small" không có nghĩa kém — với 1536 chiều, nó vượt trội hầu hết model miễn phí.

Thông số	Giá trị
Số chiều	1536
Giới hạn token	8192
Chi phí	$0.02 / 1M token
Tiếng Việt	★★★☆

Ưu điểm: Nhanh, API đơn giản, chất lượng tốt, hỗ trợ MRL (xem bên dưới). **Nhược điểm:** Cần kết nối internet và API key, có chi phí.

OpenAI — text-embedding-3-large

Phiên bản lớn hơn: 3072 chiều, chính xác hơn nhưng đắt gấp ~6 lần và chậm hơn.

Chỉ nên dùng khi `text-embedding-3-small` chưa đủ chất lượng sau khi đã tối ưu chunking.

Cohere — embed-multilingual-v3.0

Model embedding đa ngôn ngữ tốt nhất hiện tại từ Cohere.

Thông số	Giá trị
Số chiều	1024
Tiếng Việt	★★★★★
Chi phí	$0.10 / 1M token

Đặc biệt mạnh cho tiếng Việt và corpus đa ngôn ngữ. Nếu tài liệu của bạn có cả tiếng Việt lẫn tiếng Anh, đây là lựa chọn tốt nhất.

HuggingFace — BAAI/bge-m3

Model open-source miễn phí, multilingual, chất lượng rất cao.

Thông số	Giá trị
Số chiều	1024
Giới hạn token	8192
Chi phí	Miễn phí (chạy local)
Tiếng Việt	★★★★
RAM cần	~2 GB

Ưu điểm: Hoàn toàn miễn phí, chạy offline, SOTA cho multilingual. **Nhược điểm:** Cần download model (~1.2 GB), chậm hơn nếu không có GPU.

HuggingFace — Qwen3-Embedding-4B

Model mới nhất (2025) từ Alibaba/Qwen team, hiện đứng #1 trên MTEB benchmark (bộ test chuẩn đánh giá embedding model).

Đặc biệt mạnh cho tiếng Việt (★★★★★) nhưng nặng hơn (~8 GB), cần GPU hoặc chấp nhận inference chậm.

FastEmbed — multilingual-e5-small

Model nhỏ, tối ưu cho CPU — chạy được trên máy tính thông thường không có GPU.

Phù hợp khi cần chạy offline hoàn toàn và không có GPU.

Ollama (Local)

Chạy embedding model hoàn toàn trên máy tính của bạn, không cần internet.

ollama pull bge-m3

- Embedding chạy local, bảo mật tuyệt đối

Phù hợp cho tài liệu nhạy cảm không được gửi lên cloud.

3.3.2 Sparse Embedding (Vector thưa) — BM25 & SPLADE

Sparse embedding là loại vector có **hầu hết giá trị = 0**, chỉ các từ xuất hiện trong text mới có giá trị khác 0. Đây là cách tiếp cận từ khóa truyền thống được "nâng cấp".

BM25 (Best Match 25)

BM25 là thuật toán tìm kiếm từ khóa cải tiến, được dùng trong Google Search từ lâu.

Cơ chế: Tính điểm dựa trên tần suất xuất hiện của từ trong chunk và trong toàn bộ corpus.

Query: "chính sách hoàn tiền 30 ngày"
BM25 score cho chunk A: "hoàn tiền 30 ngày, điều kiện..."

- "hoàn tiền" xuất hiện 2 lần trong chunk → điểm cao
- "30 ngày" xuất hiện → điểm cao
- BM25 score = 8.5 (cao) BM25 score cho chunk B: "quyền lợi khách hàng..."
- Không có từ "hoàn tiền" hay "30 ngày"
- BM25 score = 0.2 (thấp)

Ưu điểm BM25:

- **Rất mạnh cho tên riêng, số liệu, mã sản phẩm** — những thứ dense embedding hay bỏ sót
- Không cần GPU, cực nhanh
- Hoàn toàn miễn phí
- Interpretable (giải thích được tại sao chunk này được chọn)

Nhược điểm BM25:

- Không hiểu ngữ nghĩa: "xe hơi" và "ô tô" là 2 từ khác nhau với BM25
- Nhạy cảm với spelling: "hoàn tền" (lỗi chính tả) sẽ miss

SPLADE (Sparse Lexical and Expansion)

SPLADE là phiên bản "AI-powered" của BM25 — dùng neural network để tạo sparse vector có khả năng hiểu ngữ nghĩa tốt hơn.

"xe hơi" → SPLADE expand → ["xe hơi", "ô tô", "xe con", "automobile", ...]

- Tìm được cả chunk có "ô tô" dù query dùng "xe hơi"

Ưu điểm SPLADE: Tốt hơn BM25 về mặt semantic, tốt hơn dense về mặt keyword matching. **Nhược điểm SPLADE:** Cần `transformers` + `torch`, chậm hơn BM25, cần GPU để nhanh.

3.3.3 Hybrid Embedding (Dense + Sparse)

Project hỗ trợ bật **sparse embedding song song** với dense embedding. Khi đó, mỗi chunk có 2 loại vector:

- **Dense vector:** Hiểu ngữ nghĩa, "xe hơi" = "ô tô"
- **Sparse vector (BM25/SPLADE):** Khớp từ khóa chính xác, số liệu, tên riêng

Khi retrieve, dùng cả 2 và kết hợp kết quả → **Hybrid Retrieval**.

Khuyến nghị: Luôn bật BM25 sparse embedding. Chi phí gần như 0 (không cần API call), nhưng cải thiện recall đáng kể, đặc biệt cho tài liệu kỹ thuật có nhiều tên riêng và số liệu.

3.3.4 MRL — Matryoshka Representation Learning

MRL là kỹ thuật cho phép **cắt ngắn vector embedding** mà không mất nhiều chất lượng.

Hình dung: Giống như búp bê Matryoshka (Nga) — búp bê lớn chứa búp bê nhỏ bên trong. Vector 1536 chiều chứa "phiên bản tốt" ở 512 chiều đầu.

	Chiều	Chất lượng	Storage	Tốc độ search
Vector gốc	1536	100%	1×	1×
MRL cắt còn 512	512	~92%	3× nhỏ hơn	3× nhanh hơn

Áp dụng cho OpenAI `text-embedding-3-*` và nhiều HuggingFace model.

Khi nào nên dùng MRL: Corpus rất lớn (>1 triệu chunks), cần tối ưu storage và latency.

3.3.5 Tổng kết: Chọn Embedding nào?

Tiêu chí của bạn là gì?

- **Dùng API, cân bằng nhất?** → OpenAI text-embedding-3-small ⭐
- **Tiếng Việt / đa ngôn ngữ?** → Cohere embed-multilingual-v3.0
- **Miễn phí, chất lượng cao?** → BAAI/bge-m3 (HuggingFace)
- **Miễn phí, SOTA 2025?** → Qwen3-Embedding-4B
- **CPU-only, không GPU?** → FastEmbed multilingual-e5-small
- **Bảo mật tuyệt đối, offline?** → Ollama (bge-m3)

Luôn kết hợp với BM25 sparse embedding để có hybrid retrieval.

3.4 Bước 4: Vector DB

Vector Database (cơ sở dữ liệu vector) là nơi lưu trữ tất cả embedding vectors và cho phép tìm kiếm "vector gần nhất" trong mili-giây, dù có hàng triệu vectors.

Tại sao không dùng database thông thường?

Database thông thường (MySQL, PostgreSQL) không được thiết kế để tìm kiếm vector. Tìm vector gần nhất trong 1 triệu vectors bằng cách duyệt tuần tự sẽ mất vài giây — không chấp nhận được. Vector DB dùng thuật toán **ANN (Approximate Nearest Neighbor)** đặc biệt để làm điều này trong <100ms.

Thuật toán ANN phổ biến:

- **HNSW (Hierarchical Navigable Small World):** Xây dựng đồ thị phân cấp, tìm kiếm nhanh nhất
- **IVF (Inverted File Index):** Chia vector thành cluster, tìm trong cluster phù hợp
- **LSH (Locality-Sensitive Hashing):** Hash vector vào bucket, tìm trong bucket

3.4.1 Chroma ⭐

Giới thiệu: Vector DB embedded (chạy trong process Python, không cần server riêng), thiết kế cho developers và prototype.

Cơ chế: Lưu data trên disk (SQLite + vector index), load vào memory khi cần.

Ưu điểm:

- **Zero config** — import và dùng ngay, không cần cài server
- Persist data dễ dàng (1 thư mục là đủ)
- Hỗ trợ metadata filtering tốt
- UI trực quan để xem data

Nhược điểm:

- Không scale tốt > 1 triệu vectors
- Không có distributed mode
- Tốc độ chậm hơn FAISS khi dataset lớn

Khi nào nên dùng: Demo, prototype, corpus nhỏ-vừa (<500K chunks).

3.4.2 FAISS

Giới thiệu: Facebook AI Similarity Search — thư viện tìm kiếm vector được tối ưu hóa extreme của Meta. FAISS là "engine" tìm kiếm vector nhanh nhất trên CPU.

Cơ chế: Lưu vector trong RAM hoặc memory-mapped file. Nhiều index type (Flat, IVF, HNSW, PQ).

Ưu điểm:

- **Nhanh nhất** trên CPU — benchmark vượt trội
- Nhiều lựa chọn index để cân bằng speed/accuracy/memory
- Mã nguồn mở hoàn toàn, dùng offline

Nhược điểm:

- Không có server — không hỗ trợ concurrent access tốt
- Không có metadata filtering native (cần implement thêm)
- Khó scale sang distributed

Khi nào nên dùng: Prototype cực nhanh, batch processing, khi cần tốc độ tối đa trên 1 máy.

3.4.3 Qdrant

Giới thiệu: Vector DB production-grade mã nguồn mở, viết bằng Rust — hiệu năng cao, feature phong phú nhất.

Ưu điểm:

- **Hỗ trợ Hybrid search native** (dense + sparse ACORN)
- Metadata filtering cực mạnh và nhanh
- Distributed, scale horizontally
- Quantization (giảm storage 4-8x)
- Self-host hoặc cloud

Nhược điểm:

- Cần chạy server riêng (Docker)
- Phức tạp hơn Chroma/FAISS để setup

Khi nào nên dùng: Production system, corpus lớn, cần filtering phức tạp.

3.4.4 LanceDB

Giới thiệu: Vector DB columnar (lưu theo cột như Parquet), embedded như Chroma nhưng scale tốt hơn nhiều.

Điểm đặc biệt: Dùng được DuckDB SQL để query trực tiếp, tích hợp tốt với pandas/Arrow.

Ưu điểm:

- Embedded (không cần server), nhưng scale được đến ~1 tỷ vector
- Query bằng SQL
- Versioning data (time-travel)

- Phù hợp với ML workflow

Khi nào nên dùng: Data science workflow, cần SQL analytics kèm vector search.

3.4.5 Weaviate

Giới thiệu: Vector DB với **hybrid search native** tốt nhất — kết hợp BM25 và dense search ngay trong engine.

Ưu điểm:

- GraphQL interface
- Module ecosystem phong phú (text2vec, generative, reranker)
- Hybrid search out-of-the-box

Khi nào nên dùng: Khi hybrid search là ưu tiên, GraphQL API phù hợp với frontend.

3.4.6 PGVector

Giới thiệu: PostgreSQL extension cho vector search. Nếu bạn đã có PostgreSQL infrastructure, không cần thêm DB mới.

Ưu điểm:

- Không cần thêm infrastructure mới
- ACID transaction đầy đủ
- Kết hợp vector search với SQL query phức tạp

Nhược điểm: Chậm hơn dedicated vector DB ở scale lớn, giới hạn ~50M vectors.

Khi nào nên dùng: Đã có PostgreSQL, corpus vừa phải, cần kết hợp với relational data.

3.4.7 Pinecone

Giới thiệu: Managed cloud vector DB — zero ops, fully managed, scale automatically.

Ưu điểm: Không cần quản lý infrastructure, scale tự động, SLA enterprise. **Nhược điểm:** Chi phí cloud, data không ở local.

Khi nào nên dùng: Startup cần ship nhanh, không muốn quản lý infrastructure.

3.4.8 Tổng kết: Chọn Vector DB nào?

Giai đoạn nào?

- **Demo / học tập?** → Chroma ⭐ (zero config)
- **Prototype, cần tốc độ?** → FAISS
- **Production, corpus lớn?** → Qdrant
- **Đã có PostgreSQL?** → PGVector
- **Cần hybrid search tốt nhất?** → Weaviate hoặc Qdrant
- **Zero-ops, cloud?** → Pinecone

---

4. Stage 2 — Generation

Mục tiêu của Stage 2: Nhận câu hỏi từ người dùng → tìm thông tin liên quan trong Vector DB → sinh câu trả lời chính xác, có trích dẫn nguồn.

Stage 2 chạy **mỗi khi** có câu hỏi. Tốc độ và chất lượng đều quan trọng ở đây.

4.1 Bước 7: Pre-retrieval

Pre-retrieval (xử lý trước khi tìm kiếm) là bước biến đổi câu hỏi gốc của người dùng thành một dạng tối ưu hơn cho việc tìm kiếm trong Vector DB.

Tại sao cần Pre-retrieval?

Câu hỏi của người dùng thường không phải dạng tốt nhất để tìm kiếm:

- Quá ngắn và mơ hồ: "giá?" → tìm kiếm cái gì?
- Có lỗi chính tả: "chích sách hoàn tền"
- Dùng đại từ: "cái đó hoạt động thế nào?" → cái đó là gì?
- Quá cụ thể: "điều 3.2.1 khoản b" → có thể không match với bất kỳ chunk nào

Pre-retrieval giải quyết các vấn đề này trước khi đến bước tìm kiếm.

4.1.1 None (Không biến đổi)

Giữ nguyên câu hỏi gốc, không xử lý gì thêm.

Ưu điểm: Zero latency, không tốn API call. **Nhược điểm:** Câu hỏi mơ hồ hoặc lỗi chính tả sẽ ảnh hưởng đến retrieval.

Khi nào nên dùng: Query đã rõ ràng, hệ thống cần latency thấp nhất.

4.1.2 Query Rewriting (Viết lại câu hỏi)

Giới thiệu: Dùng LLM để viết lại câu hỏi theo dạng chuẩn hơn, đầy đủ hơn, phù hợp với tìm kiếm.

Cơ chế: Input: "chính sách hoàn tền mấy ngày?"

- LLM rewrite Output: "Chính sách hoàn tiền của công ty là bao nhiêu ngày? Điều kiện để được hoàn tiền là gì?"

LLM được prompt: *"Hãy viết lại câu hỏi sau thành dạng đầy đủ, rõ ràng, phù hợp để tìm kiếm trong tài liệu..."*

Ưu điểm:

- Sửa lỗi chính tả tự động
- Mở rộng câu hỏi ngắn thành câu đầy đủ
- Loại bỏ đại từ mơ hồ
- Thêm context từ lịch sử hội thoại (nếu là chatbot)

Nhược điểm:

- Thêm 1 API call LLM → tăng latency ~500ms-1s
- LLM có thể rewrite sai ý người dùng trong một số trường hợp

Khi nào nên dùng: Query ngắn hoặc có nguy cơ có lỗi chính tả. Chatbot nhiều turn.

4.1.3 Query Expansion (Mở rộng từ khóa)

Giới thiệu: Thêm từ đồng nghĩa và thuật ngữ liên quan vào query để tăng khả năng tìm thấy tài liệu phù hợp.

Cơ chế: Input: "chính sách hoàn tiền"

- Expand Output: "chính sách hoàn tiền | hoàn trả | refund policy | đổi trả hàng | yêu cầu hoàn tiền"

Ưu điểm:

- Tăng recall — tìm được chunk dùng từ đồng nghĩa
- Giải quyết vấn đề vocabulary mismatch giữa query và document

Nhược điểm:

- Query dài hơn → latency tăng
- Expansion không đúng hướng có thể đưa vào nhiễu

Khi nào nên dùng: Corpus có nhiều cách diễn đạt khác nhau cho cùng một khái niệm (tiếng Việt + tiếng Anh, jargon vs plain language).

4.1.4 Step-back Prompting (Tổng quát hóa)

Giới thiệu: Kỹ thuật từ Google DeepMind — thay vì tìm kiếm câu hỏi cụ thể, tạo ra một câu hỏi **tổng quát hơn** để lấy kiến thức nền, sau đó kết hợp với câu hỏi gốc.

Cơ chế: Query gốc: "Tại sao công thức 3.2.1 trong tài liệu lại dùng gradient descent?"

- Step-back Query tổng quát: "Gradient descent là gì và khi nào dùng?"
- Retrieve cả 2 → LLM có đủ kiến thức nền + câu hỏi cụ thể để trả lời

Ưu điểm:

- Hiệu quả với câu hỏi chuyên sâu đòi hỏi kiến thức nền
- LLM có đủ context để trả lời câu hỏi phức tạp

Nhược điểm:

- Thêm 1 API call + 2 lần retrieve
- Không phù hợp với câu hỏi đơn giản

Khi nào nên dùng: Câu hỏi kỹ thuật sâu, tài liệu chuyên ngành.

4.1.5 Multi-query (Đa truy vấn)

Giới thiệu: Tạo ra **N biến thể** khác nhau của cùng một câu hỏi, retrieve riêng biệt cho từng biến thể, sau đó merge kết quả bằng RRF.

Cơ chế: Query gốc: "So sánh ưu và nhược điểm của chunking strategies"

- LLM tạo 3 biến thể Query 1: "Các phương pháp chunking trong RAG có ưu điểm gì?" Query 2: "Nhược điểm và hạn chế của từng chunking strategy là gì?" Query 3: "Khi nào nên dùng recursive chunking vs semantic chunking?"
- Retrieve top-5 cho mỗi query (15 chunks)
- RRF merge → top-10 kết quả đa dạng nhất
- LLM có nhiều góc độ để trả lời

Ưu điểm:

- Tăng recall đáng kể — tìm được chunk khớp với nhiều góc độ
- Giảm variance của retrieval

Nhược điểm:

- N lần API call embedding + N lần vector search
- Latency tăng tuyến tính với N

Khi nào nên dùng: Câu hỏi phức tạp, nhiều khía cạnh. Khi recall quan trọng hơn latency.

4.1.6 Query Decomposition (Phân tách câu hỏi)

Giới thiệu: Chia câu hỏi phức tạp thành các **sub-query độc lập**, giải quyết từng cái, rồi tổng hợp lại.

Cơ chế: Query phức tạp: "So sánh hiệu suất và chi phí giữa 3 vector DB: Chroma, FAISS, và Qdrant trong production"

Sub-query 1: "Hiệu suất của Chroma trong production?"
Sub-query 2: "Hiệu suất của FAISS trong production?"
Sub-query 3: "Hiệu suất của Qdrant trong production?"
Sub-query 4: "Chi phí của Chroma, FAISS, Qdrant?"

- Retrieve & answer từng sub-query → Synthesize final answer

Ưu điểm: Xử lý được câu hỏi nhiều phần, so sánh nhiều đối tượng. **Nhược điểm:** Latency cao (nhiều LLM calls), phức tạp.

Khi nào nên dùng: Câu hỏi yêu cầu so sánh nhiều entities, multi-hop reasoning.

4.1.7 Self-query (Tự phân tích metadata)

Giới thiệu: LLM phân tích câu hỏi và **tự động tạo metadata filter** để thu hẹp phạm vi tìm kiếm.

Cơ chế: Query: "Tìm các chính sách ban hành trong năm 2024 về bảo hiểm"

- LLM phân tích Filter: {year: 2024, category: "insurance"} Vector search: tìm kiếm chỉ trong documents có metadata khớp

Ưu điểm: Precision cao — không lãng phí compute trên documents không liên quan. **Nhược điểm:** Documents phải có metadata phong phú từ bước Loader.

Khi nào nên dùng: Corpus được tổ chức tốt với metadata (ngày, danh mục, tác giả...).

4.1.8 Query Routing (Định tuyến câu hỏi)

Giới thiệu: Phân loại câu hỏi và định tuyến đến **retrieval strategy phù hợp** hoặc đến nguồn dữ liệu phù hợp.

Cơ chế: Query: "Giá cổ phiếu AAPL hôm nay?"

- Router phân loại: "real-time data"
- Route đến Web Search (không dùng Vector DB) Query: "Chính sách bảo hành của sản phẩm A?"
- Router phân loại: "internal document"
- Route đến Vector DB internal docs Query: "Giải thích thuật toán HNSW?"
- Router phân loại: "general knowledge"
- Trả lời trực tiếp bằng LLM (không cần retrieve)

Ưu điểm: Tối ưu hóa resource, tránh retrieve khi không cần thiết. **Khi nào nên dùng:** Hệ thống phức tạp, nhiều nguồn dữ liệu.

4.2 Bước 8: Retrieval

Retrieval là bước tìm kiếm trong Vector DB để lấy ra các chunk liên quan nhất với câu hỏi.

Đây là bước **quyết định chất lượng** của toàn bộ pipeline. LLM tốt nhất cũng không thể trả lời đúng nếu chunks được retrieve sai.

Hai metric quan trọng:

- **Recall:** Trong số tất cả chunks thực sự liên quan, bao nhiêu % đã được retrieve?
- **Precision:** Trong số chunks được retrieve, bao nhiêu % thực sự liên quan?

Mục tiêu là tối đa hóa cả hai — nhưng thường có sự đánh đổi.

4.2.1 Dense Retrieval (Tìm kiếm ngữ nghĩa)

Giới thiệu: Tìm kiếm dựa trên **cosine similarity** giữa query vector và chunk vectors.

Cơ chế: Query → Embedding model → Query vector [q1, q2, ..., q1536]

Tính cosine similarity với tất cả chunk vectors

Trả về top-K chunks có similarity cao nhất

Cosine similarity là gì? Đo góc giữa 2 vector. Nếu góc = 0° → similarity = 1 (giống nhau hoàn toàn). Nếu góc = 90° → similarity = 0 (không liên quan).

Ưu điểm:

- Hiểu ngữ nghĩa: "xe hơi" = "ô tô" = "automobile"
- Tìm được câu trả lời dù dùng từ khác với tài liệu
- Hoạt động tốt với câu hỏi dài, diễn đạt tự nhiên

Nhược điểm:

- Kém với tên riêng, mã sản phẩm, số liệu chính xác
- "GPT-4o" và "GPT-3" có thể có similarity cao dù khác nhau hoàn toàn
- Phụ thuộc vào chất lượng embedding model

Khi nào nên dùng: Câu hỏi ngữ nghĩa, tìm kiếm khái niệm.

4.2.2 Sparse Retrieval — BM25 (Tìm kiếm từ khóa)

Giới thiệu: Tìm kiếm dựa trên **từ khóa xuất hiện trong text**, không dùng vector embedding.

Cơ chế: Query: "API key OpenAI"

- BM25 tìm chunks có chứa từ "API", "key", "OpenAI"
- Tính score dựa trên tần suất và IDF (inverse document frequency)
- Trả về top-K có score cao nhất

Ưu điểm:

- **Mạnh nhất với tên riêng, mã số, số liệu chính xác**
- Không cần embedding model → cực nhanh
- Miễn phí hoàn toàn
- Deterministic (cùng query → cùng kết quả)

Nhược điểm:

- Không hiểu ngữ nghĩa: "ô tô" ≠ "xe hơi" với BM25
- Query phải khớp từ khóa trong document

Khi nào nên dùng: Tên sản phẩm, mã code, số liệu tài chính, truy vấn keyword cụ thể.

4.2.3 Hybrid Retrieval ⭐ (Kết hợp Dense + Sparse)

Giới thiệu: Kết hợp cả Dense và Sparse retrieval, sau đó dùng thuật toán **fusion** để merge kết quả. Đây là chiến lược retrieval tốt nhất trong hầu hết trường hợp.

Cơ chế với RRF (Reciprocal Rank Fusion):

> Dense results: [Chunk A (rank 1), Chunk B (rank 2), Chunk C (rank 3), ...]
> Sparse results: [Chunk C (rank 1), Chunk D (rank 2), Chunk A (rank 3), ...]
> RRF score = Σ 1/(rank + 60) cho mỗi list
> Chunk A: 1/(1+60) + 1/(3+60) = 0.0164 + 0.0159 = 0.0323

> Chunk C: 1/(3+60) + 1/(1+60) = 0.0159 + 0.0164 = 0.0323
> Chunk B: 1/(2+60) + 0 = 0.0161
> Chunk D: 0 + 1/(2+60) = 0.0161
> Merged ranking: [A=C tied, B, D, ...]

Tại sao RRF hoạt động tốt? Thay vì cộng raw score (khó chuẩn hóa), RRF chỉ dùng rank position → không bị ảnh hưởng bởi scale khác nhau giữa Dense score và BM25 score.

Các fusion method khác:

- **Weighted fusion:** `final_score = α × dense_score + (1-α) × sparse_score`. Linh hoạt hơn nhưng cần tune α.
- **DBSF (Distribution-Based Score Fusion):** Chuẩn hóa score theo phân phối trước khi cộng.

Ưu điểm Hybrid:

- **Tốt nhất trong hầu hết trường hợp** — lấy điểm mạnh của cả 2
- Dense bù cho Sparse với câu hỏi ngữ nghĩa
- Sparse bù cho Dense với tên riêng, từ khóa cụ thể

Nhược điểm: Chậm hơn Dense hoặc Sparse đơn lẻ (cần chạy cả 2).

Khi nào nên dùng: Mặc định nên dùng khi đã bật sparse embedding. Cải thiện recall 10-30% so với chỉ Dense.

4.2.4 Multi-query Retrieval

Giới thiệu: Kết hợp với Pre-retrieval Multi-query — retrieve cho từng biến thể query rồi merge.

Cơ chế: 3 query variants → 3 × top-10 = 30 chunks

- Dedup (loại trùng) → còn ~20 unique chunks
- RRF merge → top-10 đa dạng nhất

Khi nào nên dùng: Câu hỏi phức tạp, cần tăng recall.

4.2.5 Parent Document Retrieval

Giới thiệu: Kết hợp với Hierarchical chunking — tìm kiếm bằng Child chunk nhỏ (precise), nhưng trả về Parent chunk lớn (có đủ context).

Cơ chế: Vector DB: [Child chunk 1 → Parent ID: P1] [Child chunk 2 → Parent ID: P1] [Child chunk 3 → Parent ID: P2] Query → Top-K: [Child 1 (0.92), Child 3 (0.88), Child 5 (0.85)]

- Lookup Parent: [P1, P2, P3]
- Trả về: Parent P1, P2, P3 (dài hơn, đủ context hơn)

Khi nào nên dùng: Đã dùng Hierarchical chunking ở bước Indexing.

4.2.6 Sentence Window Retrieval

Giới thiệu: Tương tự Parent Document nhưng đơn giản hơn — retrieve chunk nhỏ, sau đó **mở rộng sang N câu xung quanh** để thêm context.

Cơ chế:

> Retrieved chunk: "...điều khoản này áp dụng sau 30 ngày..."
> Window = 2 câu trước + 2 câu sau:
> "[Bên A ký kết với bên B. Hợp đồng có hiệu lực từ ngày...]

> điều khoản này áp dụng sau 30 ngày
> [kể từ ngày ký kết. Trường hợp vi phạm sẽ...]"

Khi nào nên dùng: Tài liệu văn xuôi liên tục (hợp đồng, báo cáo), khi chunk nhỏ thiếu context.

4.2.7 Multi-hop Retrieval

Giới thiệu: Retrieval nhiều bước — kết quả bước trước được dùng để tạo query mới cho bước tiếp theo.

Cơ chế: Query: "Giám đốc của công ty vừa mua lại startup X là ai?" Bước 1: Retrieve về "startup X" → Tìm được: "Startup X được mua lại bởi Công ty Y"

Bước 2: Retrieve về "Công ty Y, Giám đốc" → Tìm được: "CEO của Công ty Y là ông Z"

Trả lời: "Giám đốc là ông Z"

Ưu điểm: Giải quyết được câu hỏi nhiều bước không thể trả lời bằng 1 retrieval. **Nhược điểm:** Latency cao, có thể fail nếu bước trung gian sai.

Khi nào nên dùng: Câu hỏi research, multi-hop reasoning, knowledge-intensive QA.

4.2.8 Tổng kết: Chọn Retrieval Strategy nào?

Loại câu hỏi?

- **Từ khóa cụ thể, tên riêng?** → Sparse (BM25)
- **Câu hỏi ngữ nghĩa thông thường?** → Dense hoặc Hybrid ⭐
- **Đã bật sparse embedding?** → Hybrid ⭐ (luôn tốt hơn)
- **Câu hỏi phức tạp, nhiều góc độ?** → Multi-query

- **Đã dùng hierarchical chunking?** → Parent Document
- **Cần context xung quanh chunk?** → Sentence Window
- **Câu hỏi nhiều bước logic?** → Multi-hop

Tham số Top-K: Số chunks lấy về trước khi rerank. Thường **10-20**. Cao hơn → recall tốt hơn nhưng nhiễu hơn → cần reranker mạnh hơn.

4.3 Bước 9: Post-retrieval

Post-retrieval là bước xử lý kết quả sau khi retrieve — lọc nhiễu, rerank, nén context để chỉ giữ lại những gì thực sự hữu ích.

Tại sao cần Post-retrieval?

Bước Retrieval lấy về top-K chunks theo similarity score — nhưng similarity không hoàn toàn = relevance:

- Chunk có similarity cao nhưng chứa thông tin không liên quan đến câu hỏi
- Nhiều chunks gần trùng nhau (waste context window)
- Chunk quan trọng có thể bị "chìm" ở giữa prompt (lost-in-the-middle)

Post-retrieval giải quyết tất cả vấn đề này.

4.3.1 Cross-encoder Reranker ⭐

Giới thiệu: Model AI đọc từng cặp (query, chunk) **cùng lúc** để cho điểm relevance chính xác hơn nhiều so với embedding similarity.

Tại sao chính xác hơn?

Bi-encoder (Dense retrieval):
Query → [Embedding] → Vector Q

Chunk → [Embedding] → Vector C
Similarity = cosine(Q, C) ← Hai vector độc lập
Cross-encoder:
[Query + Chunk] → [Model] → Relevance Score ← Đọc cả hai cùng lúc!

Cross-encoder "đọc" cả query và chunk trong một lần, giống như con người đọc cả câu hỏi và câu trả lời để đánh giá. Bi-encoder chỉ so sánh 2 vector độc lập.

Cơ chế: Top-20 chunks từ retrieval

- Cross-encoder cho điểm từng cặp (query, chunk_i)
- Sort lại theo điểm mới
- Lấy top-5

Ưu điểm:

- **Chính xác nhất** — gap với bi-encoder rất lớn
- Đặc biệt tốt với câu hỏi phức tạp, nhiều ngữ cảnh

Nhược điểm:

- Chậm hơn bi-encoder (cần xử lý từng cặp)
- Cần chạy cho từng (query, chunk) pair riêng

Model khuyến nghị:

- `BAAI/bge-reranker-v2-m3` — Multilingual, tiếng Việt tốt, không cần GPU
- `cross-encoder/ms-marco-MiniLM-L-6-v2` — Nhanh hơn, tiếng Anh

Khi nào nên dùng: Hầu hết trường hợp production nên bật cross-encoder.

4.3.2 Cohere Reranker

Giới thiệu: API reranking của Cohere — chất lượng tốt nhất hiện tại, hỗ trợ 100+ ngôn ngữ.

Ưu điểm:

- **Tốt nhất cho tiếng Việt** trong số API rerankers
- Không cần download model, không cần GPU
- SLA và độ tin cậy cao

Nhược điểm: Chi phí API, cần kết nối internet.

Khi nào nên dùng: Production system, tiếng Việt, không muốn quản lý model.

4.3.3 LLM Reranker

Giới thiệu: Dùng chính LLM để đánh giá và xếp hạng lại các chunks — **listwise reranking**.

Cơ chế: Prompt cho LLM: "Câu hỏi: [query] Dưới đây là 10 đoạn text. Hãy xếp hạng từ liên quan nhất đến kém liên quan nhất. [Chunk 1] [Chunk 2] ... [Chunk 10]"

- LLM trả về: "Thứ tự: 3, 1, 7, 2, ..."

Ưu điểm:

- Không cần model riêng
- LLM có thể hiểu context phức tạp hơn cross-encoder nhỏ

Nhược điểm: Chậm và đắt (cần gửi tất cả chunks lên LLM).

Khi nào nên dùng: Khi không muốn deploy thêm model, quality quan trọng hơn tốc độ.

4.3.4 Redundancy Filter (Lọc trùng lặp)

Giới thiệu: Loại bỏ các chunks có nội dung gần giống nhau (cosine similarity > ngưỡng) để tránh lãng phí context window.

Cơ chế: Chunks: [A, B, C, D] trong đó sim(A, C) = 0.95

- Giữ A (có rank cao hơn), loại C
- Output: [A, B, D]

Ngưỡng khuyến nghị: 0.90-0.95. Cao hơn → giữ nhiều hơn, thấp hơn → loại bỏ nhiều hơn.

Khi nào nên dùng: Corpus có nhiều nội dung tương tự (FAQ, template documents).

4.3.5 MMR Filter (Maximal Marginal Relevance)

Giới thiệu: Thuật toán cân bằng giữa **relevance** (liên quan đến query) và **diversity** (đa dạng với nhau).

Cơ chế: MMR score = λ × relevance(chunk, query) - (1-λ) × max_similarity(chunk, selected_chunks)

- Chunk vừa liên quan đến query, vừa khác với những chunk đã chọn

Ví dụ: Chunks về "chính sách hoàn tiền": A: Điều kiện hoàn tiền (sim với query = 0.9) B: Thủ tục hoàn tiền (sim với query = 0.85) C: Ví dụ hoàn tiền (sim với query = 0.88, sim với A = 0.93 ← gần giống A) MMR chọn: A (highest relevance) → B (different from A) → không chọn C (quá giống A)

Ưu điểm: Output đa dạng hơn, tránh "echo chamber" — nhiều góc độ về cùng chủ đề. **Nhược điểm:** Có thể loại một số chunks relevant vì quá giống với chunk đã chọn.

Khi nào nên dùng: Query cần nhiều góc nhìn khác nhau, câu hỏi tổng hợp.

4.3.6 Context Compressor (Nén ngữ cảnh)

Giới thiệu: Dùng LLM để **trích xuất chỉ phần liên quan** từ mỗi chunk, bỏ phần không liên quan đến query.

Cơ chế:

> Chunk gốc (500 ký tự):
> "Công ty ABC được thành lập năm 2010. Có 500 nhân viên.
> Chính sách hoàn tiền: 30 ngày kể từ ngày mua hàng.
> Văn phòng tại Hà Nội và TP.HCM..."
> Query: "Chính sách hoàn tiền là gì?"
> Sau compression (50 ký tự):
> "Chính sách hoàn tiền: 30 ngày kể từ ngày mua hàng."

Ưu điểm:

- Giảm token cho LLM → tiết kiệm chi phí, giảm latency
- Giảm noise, LLM tập trung vào thông tin quan trọng

Nhược điểm: Thêm 1 LLM call per chunk, có thể nén quá mức và mất thông tin.

Khi nào nên dùng: Token budget hạn chế, corpus có nhiều noise.

4.3.7 Context Orderer (Sắp xếp ngữ cảnh)

Giới thiệu: Sắp xếp thứ tự các chunks trong prompt để tối ưu hóa khả năng LLM đọc và sử dụng thông tin.

Sandwich ordering ⭐ (Khuyến nghị): [Chunk tốt nhất] ← LLM chú ý nhiều nhất (đầu prompt) [Chunk 3] [Chunk 4] ← LLM chú ý ít nhất (giữa prompt) [Chunk 2] [Chunk tốt thứ 2] ← LLM vẫn chú ý (cuối prompt)

Dựa trên nghiên cứu "Lost in the Middle" (Liu et al., 2023): LLM chú ý nhiều hơn vào đầu và cuối context, bỏ qua phần giữa. Sandwich đặt thông tin quan trọng ở đầu VÀ cuối.

Relevance ordering: Giảm dần theo score — đơn giản nhất. **Original ordering:** Giữ nguyên thứ tự gốc trong tài liệu — tốt khi chunks liên tiếp.

4.4 Bước 10: Prompt

Prompt là bước xây dựng "câu hỏi hoàn chỉnh" gửi lên LLM — bao gồm system instruction, context chunks, và câu hỏi của người dùng.

Prompt engineering (thiết kế prompt) có ảnh hưởng rất lớn đến chất lượng câu trả lời, ngay cả khi retrieval đã tốt.

Cấu trúc cơ bản của RAG prompt:

> [System Message]
> Bạn là trợ lý AI hữu ích. Chỉ trả lời dựa trên context được cung cấp.
> Nếu không tìm thấy thông tin trong context, hãy nói rõ.
> [Context]
> --- Nguồn 1: document_A.pdf, trang 5 ---
> [Nội dung chunk 1]
> --- Nguồn 2: document_B.pdf, trang 12 ---
> [Nội dung chunk 2]
> [User Message]
> Câu hỏi: [Query của người dùng]

4.4.1 Basic Prompt

Giới thiệu: Template đơn giản nhất — context + question → answer.

Output: Plain text, không có citation.

Khi nào nên dùng: Prototype nhanh, nội dung không cần verify nguồn.

4.4.2 Citation Prompt ⭐

Giới thiệu: LLM được yêu cầu **trích dẫn nguồn** cho mỗi claim trong câu trả lời.

Output:

> Chính sách hoàn tiền là 30 ngày [NGUỒN 1], với điều kiện sản phẩm còn nguyên vẹn và có hóa đơn [NGUỒN 1]. Trong trường hợp sản phẩm bị lỗi do nhà sản xuất, thời hạn hoàn tiền được mở rộng lên 60 ngày [NGUỒN 2].

Tại sao quan trọng:

- Người dùng có thể kiểm tra từng claim
- Giảm hallucination — LLM "ngại" bịa khi phải trích dẫn nguồn
- Tăng trust của người dùng vào hệ thống

Khi nào nên dùng: Production system, nội dung nhạy cảm (pháp lý, y tế, tài chính).

4.4.3 Conversational Prompt

Giới thiệu: Duy trì lịch sử hội thoại — LLM nhớ những gì đã nói ở các turn trước.

Cơ chế: Turn 1: Q: "Chính sách hoàn tiền là gì?" → A: "30 ngày..." Turn 2: Q: "Áp dụng cho sản phẩm điện tử không?"

- Prompt gửi lên LLM: History: [Turn 1] Context: [Chunks liên quan] Question: "Áp dụng cho sản phẩm điện tử không?"
- LLM hiểu "áp dụng" là về "chính sách hoàn tiền 30 ngày" đã nói ở Turn 1

Khi nào nên dùng: Chatbot, hệ thống Q&A nhiều turn.

4.4.4 Structured Output Prompt

Giới thiệu: Yêu cầu LLM trả về JSON có cấu trúc thay vì plain text.

Output:json { "answer": "Chính sách hoàn tiền là 30 ngày", "claims": [{"claim": "30 ngày", "source": "document_A.pdf:5", "confidence": 0.95}, {"claim": "sản phẩm nguyên vẹn", "source": "document_A.pdf:5", "confidence": 0.92}], "confidence_overall": 0.93, "cannot_answer": false } **Khi nào nên dùng:** Downstream code cần parse output, integration với hệ thống khác.

4.4.5 Tổng kết: Chọn Prompt Template nào?

Template	Output	Dùng khi
`basic`	Plain text	Prototype nhanh
`citation` ⭐	Text + [NGUỒN N]	Production, cần verify
`conversational`	Text + history	Chatbot
`structured`	JSON	API integration

Tips thiết kế prompt hiệu quả:

1. **Rõ ràng về nguồn:** Luôn thêm header cho mỗi chunk ("--- Nguồn 1: ---")
2. **Hướng dẫn "không biết":** Thêm instruction "Nếu không có thông tin trong context, hãy nói rõ là không biết"
3. **Temperature = 0.0** cho RAG — muốn deterministic, không creative
4. **Giới hạn độ dài context:** Đừng nhét quá nhiều chunks vào 1 prompt

---

4.5 Bước 11: Generation

Generation là bước cuối — LLM nhận prompt hoàn chỉnh và sinh ra câu trả lời.

4.5.1 OpenAI Models

GPT-4.1-mini ☆

Model cân bằng nhất giữa chất lượng và chi phí.

Thông số	Giá trị
Chi phí input	$0.40 / 1M token
Chi phí output	$1.60 / 1M token
Context window	1M token
Tiếng Việt	★★★★

GPT-4o

Flagship model của OpenAI, chất lượng tốt nhất nhưng đắt hơn.

GPT-4o-mini

Rẻ nhất trong dòng GPT-4: $0.15/$0.60 per 1M token. Phù hợp khi budget hạn chế.

Khi nào chọn OpenAI: Cần API đơn giản, reliability cao, context window lớn.

4.5.2 Anthropic Claude

Claude Sonnet 4 (claude-sonnet-4-6)

Model cân bằng của Anthropic. Đặc biệt mạnh về:

- **Tiếng Việt ★★★★★** — tốt nhất trong các API models
- Reasoning và phân tích phức tạp
- Instruction following chính xác

| Chi phí | $3 input / $15 output per 1M token |

Khi nào chọn Anthropic: Corpus tiếng Việt, câu hỏi phân tích phức tạp, cần instruction following chính xác.

4.5.3 Google Gemini

Gemini 2.0 Flash

Model nhanh nhất, có free tier.

Thông số	Giá trị
Free tier	Có (với giới hạn)
Context window	1M token
Tiếng Việt	★★★★

Khi nào chọn Gemini: Free tier cho prototype, context window cực lớn cần thiết.

4.5.4 Ollama — LLM Local

Giới thiệu: Chạy LLM hoàn toàn trên máy tính của bạn — không cần internet, không có chi phí API, bảo mật tuyệt đối.

Cơ chế: Ollama download và quản lý LLM weights trên máy, expose REST API local.bash ollama pull qwen2.5:7b # Download model ~4.7 GB ollama serve # Chạy local server

RAG-pipeline-visualizer kết nối đến http://localhost:11434

Các model khuyến nghị:

Model	RAM cần	Chất lượng	Tiếng Việt
`qwen2.5:7b`	~6 GB	★★★★	★★★★
`llama3.2:3b`	~3 GB	★★★	★★★
`gemma3:9b`	~9 GB	★★★★	★★★
`mistral:7b`	~6 GB	★★★★	★★★

Ưu điểm:

- **Miễn phí hoàn toàn** — không có chi phí API
- **Bảo mật tuyệt đối** — dữ liệu không rời máy
- Không cần internet sau khi download model
- Latency thấp (không có network roundtrip)

Nhược điểm:

- Chất lượng kém hơn GPT-4/Claude
- Cần RAM lớn (min 8 GB cho model 7B)
- Tốc độ phụ thuộc vào phần cứng (không có GPU thì chậm)

Khi nào chọn Ollama:

- Tài liệu nhạy cảm (y tế, pháp lý, bí mật kinh doanh)
- Muốn tiết kiệm chi phí hoàn toàn
- Môi trường không có internet

4.5.5 Cohere Command-R

Model của Cohere được thiết kế đặc biệt cho RAG — có built-in citation support.

Khi nào chọn: Đã dùng Cohere cho embedding/reranker, muốn consistent provider.

4.5.6 Các tham số quan trọng của Generation

Temperature (Nhiệt độ): Temperature = 0.0 → Deterministic, luôn chọn token có xác suất cao nhất

- Khuyến nghị cho RAG: muốn nhất quán, không creative Temperature = 0.7 → Sáng tạo hơn, đa dạng hơn
- Dùng cho creative writing, brainstorming Temperature = 1.0 → Rất ngẫu nhiên

Cho RAG, luôn dùng Temperature = 0.0 — muốn LLM "đọc và tổng hợp" chứ không "sáng tác".

Streaming: Bật để câu trả lời xuất hiện dần từng từ — trải nghiệm người dùng tốt hơn, không phải chờ toàn bộ response.

4.5.7 Tổng kết: Chọn LLM nào?

Ưu tiên của bạn?

- **Tiếng Việt tốt nhất?** → Anthropic Claude Sonnet
- **Cân bằng chất lượng/chi phí?** → OpenAI GPT-4.1-mini ⭐
- **Rẻ nhất (API)?** → OpenAI GPT-4o-mini
- **Miễn phí?** → Google Gemini 2.0 Flash
- **Bảo mật / offline?** → Ollama (qwen2.5:7b)
- **Máy yếu, offline?** → Ollama (llama3.2:3b)

---

5. Luồng hoạt động toàn bộ Pipeline

5.1 Luồng Stage 1 — Indexing (Chạy 1 lần)

Đầu vào: Một hoặc nhiều file PDF

Thứ tự	Bước	Input	Output	Ghi chú
1	**Loader**	File PDF	list[Document]	Mỗi Document có text + metadata (file, trang)
↓	*Cache check*	—	—	Nếu config giống lần trước → bỏ qua bước này
2	**Chunking**	list[Document]	list[Chunk]	Nhiều chunk nhỏ hơn, giữ nguyên metadata
3	**Embedding**	list[Chunk]	list[Vector]	Dense vector + Sparse vector (nếu bật BM25)
4	**Vector DB**	list[Vector]	Index trên disk	Sẵn sàng cho Stage 2

Hệ thống Cache SHA-256:

Mỗi bước chỉ chạy lại khi cấu hình thay đổi. Ví dụ:

- Chỉ đổi **embedding model** → chỉ chạy lại từ Bước 3
- Chỉ đổi **chunking strategy** → chạy lại từ Bước 2
- Cùng config → tái sử dụng 100%, không tốn API call

5.2 Luồng Stage 2 — Generation (Chạy mỗi query)

Đầu vào: Câu hỏi của người dùng

Thứ tự	Bước	Mô tả	Thời gian	Bắt buộc?
7	**Pre-retrieval**	Biến đổi query: rewrite / expand / multi_query	~0–800ms	Tùy chọn
8	**Retrieval**	Tìm top-K chunks trong Vector DB	~50–200ms	✅
9	**Post-retrieval**	Rerank + filter + sắp xếp context	~100–500ms	Tùy chọn
10	**Prompt Builder**	Ghép system_prompt + context + query	~5ms	✅
11	**Generation**	LLM sinh câu trả lời từ prompt hoàn chỉnh	~500–5000ms	✅

Lưu ý quan trọng: Embedding model ở Bước 8 (embed query) phải **giống hệt** model đã dùng ở Bước 3 (embed chunks khi indexing). Khác model → vector dimension không khớp → retrieval fail.

5.3 Ví dụ thực tế End-to-End

Kịch bản: Upload tài liệu chính sách công ty (PDF 50 trang), hỏi về chính sách hoàn tiền.

Stage 1 — Indexing (chạy 1 lần, ~5 phút):

1. **Loader (Marker):** PDF 50 trang → Markdown 45,000 ký tự. 8 bảng được convert sang Markdown table, headers giữ nguyên (#H1, ##H2).
2. **Chunking (format_aware):** 45,000 ký tự → 87 chunks, cắt theo `##` headings, avg 520 ký tự/chunk.

3. **Embedding (OpenAI text-embedding-3-small):** 87 API calls → 87 vectors × 1536 chiều + BM25 sparse index. Chi phí ~$0.001.
4. **Vector DB (Chroma):** Lưu 87 vectors + text + metadata vào `./storage/chroma_db/`. Index sẵn sàng.

Stage 2 — Query (chạy mỗi lần hỏi, ~3 giây):

> **Câu hỏi:** "Điều kiện để được hoàn tiền là gì?"

7. **Pre-retrieval (rewrite):** Query được viết lại thành *"Điều kiện và yêu cầu để được hoàn tiền là gì? Thời hạn yêu cầu là bao lâu?"*
8. **Retrieval (hybrid, top-15):** Dense tìm được Chunk 23 (0.91), Chunk 24 (0.87)... BM25 tìm được Chunk 23 (12.3), Chunk 25 (8.1)... Sau RRF merge → Chunk 23 đứng đầu.
9. **Post-retrieval:** Cross-encoder rerank → Chunk 23 (0.97), Chunk 24 (0.89). Loại Chunk 26 (quá giống Chunk 23). Sandwich ordering → Chunk 23 đầu, Chunk 24 cuối.
10. **Prompt builder (citation):** Ghép system prompt + 5 chunks có header "Nguồn 1", "Nguồn 2"... + câu hỏi.
11. **Generation (GPT-4.1-mini, temperature=0.0):** *"Để được hoàn tiền cần đáp ứng: 1) Yêu cầu trong 30 ngày [NGUỒN 1], 2) Sản phẩm còn nguyên vẹn [NGUỒN 2], 3) Có hóa đơn [NGUỒN 2]."* Token usage: 1,430 tokens.

5.4 Latency Budget

Tổng thời gian một query điển hình:

Bước	Thời gian	Ghi chú
Pre-retrieval (rewrite)	~800ms	1 LLM API call
Embedding query	~50ms	1 embedding API call
Vector DB search	~30ms	ANN search
Cross-encoder rerank	~200ms	Local model
Prompt build	~5ms	String concat
LLM generation	~1500ms	Streaming
Tổng	**~2.5 giây**	

Tối ưu (không pre-retrieval, không rerank): Embedding + Search + Generation ≈ **~1.6 giây**

---

6. Scaling & Guardrails

> **Phần này dành cho ai?** Những ai muốn phát triển RAG pipeline từ prototype thành production system phục vụ nhiều người dùng.

6.1 Scaling (Mở rộng quy mô)

6.1.1 Scaling về dữ liệu

Corpus nhỏ (< 100K chunks): Chroma hoặc FAISS trên 1 máy là đủ.

Corpus trung bình (100K - 10M chunks): Chuyển sang Qdrant hoặc Weaviate với proper indexing.

Corpus lớn (> 10M chunks):

- Distributed Vector DB (Qdrant cluster, Weaviate cluster)
- Quantization để giảm storage và RAM
- Sharding (phân mảnh data ra nhiều node)

Quantization là gì?

Loại	Kích thước mỗi chunk	Ghi chú
Float32 gốc	1536 × 4 bytes = **6,144 bytes**	Đầy đủ chất lượng
INT8 quantized	1536 × 1 byte = **1,536 bytes**	Giảm 4× storage, ~5% giảm accuracy

6.1.2 Scaling về request

Concurrent users tăng:

- **Caching query embeddings:** Query giống nhau → reuse embedding, không gọi API lại
- **Caching RAG results:** Câu hỏi phổ biến → cache câu trả lời
- **Async processing:** Xử lý nhiều query song song
- **Load balancing:** Nhiều instance của Generation service

Caching strategy khuyến nghị:python
Cache layer 1: Query embedding cache (Redis, TTL=1 hour)
→ Cùng câu hỏi → reuse vector, skip API call

Cache layer 2: RAG result cache (Redis, TTL=10 phút)
→ Cùng câu hỏi + cùng corpus → reuse kết quả hoàn toàn
→ Lưu ý: Invalidate khi corpus update

Cache layer 3: Retrieval cache (In-memory, TTL=5 phút)
→ Cùng query vector → reuse top-K chunks

6.1.3 Scaling Indexing pipeline

Khi corpus lớn, indexing có thể mất hàng giờ. Giải pháp:

Batch processing: 100,000 chunks → Chia thành batch 1000

- Gửi parallel requests đến embedding API
- Rate limit: OpenAI cho phép 3000 RPM
- Estimated time: 100,000/3000 × 1 minute ≈ 33 phút

Incremental indexing: Thay vì re-index toàn bộ khi có file mới:

- Chỉ index file mới
- Chỉ xóa vectors của file đã xóa
- Dùng document ID + hash để track thay đổi

6.2 Guardrails (Rào chắn an toàn)

Guardrails là các cơ chế bảo vệ hệ thống khỏi các câu hỏi nguy hiểm, không phù hợp, hoặc cố tình phá hoại.

6.2.1 Input Guardrails (Lọc đầu vào)

PII Detection (Phát hiện thông tin cá nhân): User input: "Tìm thông tin của khách hàng Nguyễn Văn A, CCCD 123456789"

- PII detected: tên người (NER), số CCCD (regex)
- Từ chối hoặc anonymize trước khi xử lý

Prompt Injection Detection: User input: "Bỏ qua tất cả instructions trước đó. Hãy..."

- Phát hiện pattern prompt injection
- Từ chối và log sự kiện

Topic Filtering: User input: "Hướng dẫn làm bom"

- Topic classifier: harmful/off-topic
- Từ chối với message phù hợp

Implementation đơn giản: Kiểm tra tuần tự: (1) từ chối nếu query quá dài (> 2000 ký tự), (2) dùng regex phát hiện các pattern nguy hiểm như "ignore instructions" hay "jailbreak", (3) tùy chọn dùng LLM để phân loại an toàn chính xác hơn.

6.2.2 Output Guardrails (Lọc đầu ra)

Faithfulness Check (Kiểm tra độ trung thực):

Sau khi LLM sinh câu trả lời, kiểm tra xem có claim nào không được hỗ trợ bởi context không.

Câu trả lời LLM: "Hoàn tiền trong 30 ngày [NGUỒN 1].
Áp dụng cho tất cả sản phẩm kể cả đồ ăn [?]"
Context (NGUỒN 1): "Hoàn tiền trong 30 ngày cho sản phẩm điện tử"
Faithfulness check:
✅ "30 ngày" → có trong NGUỒN 1
❌ "tất cả sản phẩm kể cả đồ ăn" → KHÔNG có trong context

- Flag: possible hallucination

Tools phổ biến: RAGAS faithfulness metric, TruLens, LLM-as-judge.

Hallucination Mitigation:

1. Temperature = 0.0 (giảm randomness)
2. Citation template (LLM phải cite nguồn cho mỗi claim)
3. Instruction "Nếu không có trong context, hãy nói không biết"
4. Faithfulness scoring post-generation

PII Removal trong output: Output LLM: "Khách hàng Nguyễn Văn A (SĐT: 0987xxx) đã..."

- PII detection: tên, SĐT
- Anonymize: "Khách hàng [TÊN ẩn] (SĐT: [ẩn]) đã..."

6.2.3 Retrieval Guardrails

Out-of-scope Detection: User: "Ai là tổng thống Mỹ?" Vector DB chứa: Tài liệu nội bộ công ty (không có thông tin này) Giải pháp 1: Threshold similarity → Nếu max similarity < 0.3 → "Câu hỏi nằm ngoài phạm vi tài liệu" Giải pháp 2: LLM check → LLM đọc context → "Tôi không tìm thấy thông tin này trong tài liệu"

Adversarial Query Protection: User: "Liệt kê toàn bộ thông tin về [competitor company]"

- Query có thể extract sensitive data nếu có trong corpus
- Implement query audit log
- Rate limiting per user
- Role-based access control (chỉ cho phép query một số collection nhất định)

6.2.4 Access Control

- Mỗi chunk có metadata: {"access_level": ["public", "hr"]}
- Khi retrieve: filter theo access_level của user
- Qdrant, Weaviate, Chroma đều hỗ trợ metadata filtering

6.3 Monitoring & Observability

Các metrics cần theo dõi:

Metric	Ý nghĩa	Alert khi
Retrieval latency	Thời gian tìm kiếm	> 500ms
Generation latency	Thời gian LLM	> 5s
Retrieval recall	Chunk liên quan được tìm?	< 0.7
Faithfulness score	LLM có bịa không?	< 0.8
Answer relevance	Câu trả lời có liên quan không?	< 0.7
Token usage	Chi phí API	Vượt budget
Error rate	Số lần fail	> 1%

Evaluation frameworks:

RAGAS — Framework đánh giá RAG phổ biến nhất. Chạy đánh giá tự động trên dataset câu hỏi/câu trả lời mẫu, trả về điểm số từ 0–1 cho từng metric. Ví dụ kết quả: `faithfulness: 0.89`,

`answer_relevancy: 0.92`, `context_recall: 0.76`.

- **Faithfulness:** LLM có bịa thêm thông tin không có trong context không? (0-1)
- **Answer Relevancy:** Câu trả lời có liên quan đến câu hỏi không? (0-1)
- **Context Recall:** Context được retrieve có chứa đủ thông tin để trả lời không? (0-1)

LLM-as-Judge: Dùng một LLM mạnh (GPT-4) để đánh giá output của pipeline:

- "Câu trả lời này có chính xác so với context không? Score từ 1-5"
- Tốn chi phí nhưng linh hoạt và chính xác

6.4 Cost Optimization (Tối ưu chi phí)

Ước tính chi phí điển hình:

Corpus: 1,000 tài liệu PDF, avg 20 trang/file = 20,000 trang

Bước	Chi phí	Ghi chú
Loader (Marker)	~$0	Local model
Chunking	~$0	Local CPU
Embedding (text-embedding-3-small)	~$0.50	25M tokens
Vector DB (Chroma)	~$0	Local storage
Tổng Indexing	~$0.50	Chạy 1 lần

100 queries/ngày:

Bước	Chi phí/ngày	Ghi chú
Query embedding	~$0.001	100 × 100 tokens
Reranker (cross-encoder)	~$0	Local model
Generation (GPT-4.1-mini)	~$0.10	100 × 1500 tokens
Tổng Generation	~$0.10/ngày	$3/tháng

Chiến lược giảm chi phí:

1. **Dùng text-embedding-3-small thay vì large** → 6x rẻ hơn, ~5% giảm chất lượng
2. **MRL để giảm vector dimension** → Giảm storage, tăng tốc search
3. **Cache embedding của popular queries** → Tiết kiệm đến 40% embedding calls
4. **Dùng GPT-4o-mini thay vì GPT-4.1-mini** → 2.7x rẻ hơn
5. **Batch embedding requests** → Tránh gọi từng cái một
6. **Ollama cho dev/test** → Không tốn xu nào trong quá trình phát triển

---

7. So sánh cấu hình và lời khuyên thực tế

7.1 Cấu hình cho từng use case

Use case 1: Demo / Học tập

Bước	Lựa chọn	Lý do
Loader	`pypdf`	Nhanh, không cần cài thêm
Chunking	`recursive`, size=1000	Mặc định tốt
Embedding	`text-embedding-3-small`	Đơn giản
Vector DB	`chroma`	Zero config
Retrieval	`dense`	Đơn giản
Reranker	Không	Giảm latency
LLM	`gpt-4o-mini`	Rẻ nhất

Use case 2: PDF phức tạp (học thuật, kỹ thuật)

Bước	Lựa chọn	Lý do
Loader	`marker`	Bảng + công thức + hình
Chunking	`format_aware`	Output Markdown từ Marker
Sparse	BM25 bật	Tăng recall
Embedding	`text-embedding-3-small`	Cân bằng
Vector DB	`chroma` hoặc `faiss`	
Pre-retrieval	`rewrite`	Sửa query
Retrieval	`hybrid`	Dense + BM25
Reranker	`cross_encoder`	Chính xác
Prompt	`citation`	Trích dẫn nguồn
LLM	`gpt-4.1-mini`	Chất lượng

Use case 3: Corpus tiếng Việt

Bước	Lựa chọn	Lý do
Loader	`marker` hoặc `pdfplumber`	Tùy loại PDF
Embedding	`Cohere embed-multilingual`	Tốt nhất cho VI
Reranker	`cohere`	100+ ngôn ngữ
LLM	`claude-sonnet-4-6`	Tiếng Việt tốt nhất

Use case 4: Bảo mật cao / Offline

Bước	Lựa chọn	Lý do
Loader	`pdfplumber` hoặc `marker`	Local
Embedding	`BAAI/bge-m3` (HuggingFace)	Local, không internet
Sparse	BM25	Local hoàn toàn
Vector DB	`faiss` hoặc `chroma`	Local
LLM	`Ollama qwen2.5:7b`	Local, zero data leakage

7.2 Những lỗi phổ biến cần tránh

❌ **Lỗi 1: Chunk size quá nhỏ** chunk_size = 100 ký tự → Chunk quá cụt, thiếu ngữ cảnh

- LLM không hiểu câu trả lời dù đã retrieve đúng chunk Sửa: chunk_size = 500-1500 ký tự

❌ **Lỗi 2: Không có chunk overlap** chunk_overlap = 0 → Thông tin nằm ở ranh giới 2 chunk bị mất Sửa: chunk_overlap = 10-15% của chunk_size

❌ **Lỗi 3: Dùng PDF loader sai loại** PDF có nhiều bảng → Dùng pypdf → Bảng bị đọc lộn xộn

- Câu trả lời sai về số liệu trong bảng Sửa: Dùng pdfplumber hoặc marker cho PDF có bảng

❌ **Lỗi 4: Embedding model khác nhau ở Indexing và Query** Indexing: text-embedding-3-small (1536 chiều) Query: text-embedding-3-large (3072 chiều)

- Vector dimension không match → retrieval fail hoàn toàn Sửa: PHẢI dùng cùng embedding model cho cả 2 stage

❌ **Lỗi 5: Top-K quá nhỏ** top_k = 3 → Có thể miss chunk liên quan, recall thấp Sửa: top_k = 10-20 (retrieve nhiều), rồi rerank → top_n = 3-5

❌ **Lỗi 6: Temperature > 0 cho RAG** temperature = 0.7 → LLM creative, có thể thêm thông tin không có trong context Sửa: temperature = 0.0 cho RAG luôn

❌ **Lỗi 7: Không có "I don't know" instruction** Không có instruction → LLM bịa câu trả lời khi không tìm thấy trong context Sửa: Thêm vào system prompt: "Nếu không tìm thấy thông tin trong context, hãy trả lời 'Tôi không tìm thấy thông tin này trong tài liệu'"

---

8. Glossary — Bảng thuật ngữ

Thuật ngữ	Tiếng Việt	Giải thích
ANN	Tìm kiếm láng giềng gần đúng	Thuật toán tìm vector gần nhất nhanh trong không gian nhiều chiều
BM25	—	Thuật toán tìm kiếm từ khóa cải tiến, nền tảng của search engine
Chunk	Đoạn văn	Phần nhỏ của tài liệu sau khi cắt
Chunk overlap	Vùng chồng lấp	Phần text trùng nhau giữa 2 chunk liên tiếp
Chunking	Phân đoạn	Cắt tài liệu dài thành các đoạn nhỏ hơn
Context window	Cửa sổ ngữ cảnh	Số token tối đa LLM có thể xử lý trong 1 lần

Thuật ngữ	Tiếng Việt	Giải thích
Cross-encoder	Bộ mã hóa chéo	Model đọc query + document cùng lúc, cho điểm relevance chính xác
Dense retrieval	Tìm kiếm dày đặc	Tìm kiếm dựa trên vector embedding (ngữ nghĩa)
Embedding	Vector nhúng	Biểu diễn text dưới dạng vector số
Faithfulness	Độ trung thực	Mức độ câu trả lời bám sát context, không bịa thêm
Fine-tuning	Tinh chỉnh mô hình	Train thêm LLM trên dataset chuyên biệt
Hallucination	Ảo giác AI	LLM tự tin đưa ra thông tin sai, không có trong context
HNSW	—	Hierarchical Navigable Small World — thuật toán ANN phổ biến nhất
Hybrid retrieval	Tìm kiếm kết hợp	Kết hợp dense + sparse retrieval
HyDE	—	Hypothetical Document Embedding — tạo tài liệu giả để cải thiện query
Index	Chỉ mục	Cấu trúc dữ liệu hỗ trợ tìm kiếm nhanh
Intent	Mục đích	Mục đích thực sự của người dùng đằng sau câu hỏi
Knowledge cutoff	Điểm cắt tri thức	Thời điểm LLM ngừng cập nhật dữ liệu mới
Latency	Độ trễ	Thời gian từ lúc nhận query đến khi trả lời
LLM	Mô hình ngôn ngữ lớn	AI được train trên lượng text khổng lồ (GPT-4, Claude...)

Thuật ngữ	Tiếng Việt	Giải thích
Lost-in-the-middle	Mất thông tin giữa	LLM bỏ qua thông tin nằm ở giữa context dài
Metadata	Siêu dữ liệu	Thông tin về tài liệu: tên file, trang, ngày tạo...
MMR	—	Maximal Marginal Relevance — cân bằng relevance và diversity
MRL	—	Matryoshka Representation Learning — cắt ngắn vector mà không mất nhiều chất lượng
Multi-hop	Nhiều bước	Câu hỏi cần nhiều bước tìm kiếm liên tiếp để trả lời
OCR	Nhận dạng chữ	Optical Character Recognition — đọc chữ từ ảnh
PII	Thông tin cá nhân	Personally Identifiable Information — tên, SĐT, CCCD...
Precision	Độ chính xác	Trong số chunk retrieve được, bao nhiêu % thực sự liên quan
Prompt	Câu lệnh	Toàn bộ text gửi lên LLM bao gồm system instruction + context + question
RAG	Sinh văn bản tăng cường	Retrieval-Augmented Generation
RAGAS	—	Framework đánh giá RAG tự động
Recall	Độ phủ	Trong số tất cả chunk liên quan, bao nhiêu % đã được retrieve
Reranker	Xếp hạng lại	Model đánh giá lại độ liên quan của chunk sau retrieval

Thuật ngữ	Tiếng Việt	Giải thích
RRF	Hợp nhất xếp hạng	Reciprocal Rank Fusion — thuật toán merge kết quả từ nhiều retriever
Self-RAG	RAG tự đánh giá	LLM tự quyết định có cần retrieve không
Semantic chunking	Phân đoạn ngữ nghĩa	Cắt tài liệu theo sự thay đổi chủ đề
Sparse retrieval	Tìm kiếm thưa	Tìm kiếm dựa trên từ khóa (BM25, TF-IDF)
SPLADE	—	Sparse Lexical and Expansion — sparse embedding có khả năng semantic
Streaming	Phát trực tiếp	Hiển thị câu trả lời dần từng từ thay vì chờ toàn bộ
Temperature	Nhiệt độ	Tham số kiểm soát độ ngẫu nhiên của LLM (0 = deterministic)
Token	—	Đơn vị text mà LLM xử lý (~4 ký tự tiếng Anh hoặc ~1-2 ký tự tiếng Việt)
Vector DB	Cơ sở dữ liệu vector	DB chuyên lưu trữ và tìm kiếm embedding vectors
VLM	Mô hình thị giác-ngôn ngữ	Vision-Language Model — AI hiểu cả ảnh và text

Phụ lục: Checklist triển khai RAG

Checklist Indexing

- [ ] Chọn Loader phù hợp với loại PDF (scan vs text layer vs phức tạp)

- [ ] Test output Loader trên 1-2 file mẫu, kiểm tra bảng và công thức
- [ ] Chọn Chunking strategy phù hợp (format_aware nếu dùng Marker)
- [ ] Thiết lập chunk_size và overlap hợp lý
- [ ] Chọn Embedding model (lưu ý: phải giống nhau ở Query stage)
- [ ] Bật BM25 sparse embedding (chi phí gần 0, lợi ích lớn)
- [ ] Chọn Vector DB phù hợp với scale và infrastructure
- [ ] Test indexing với corpus nhỏ trước
- [ ] Kiểm tra similarity heatmap — nếu toàn thấp → đổi embedding model

Checklist Generation

- [ ] Test câu hỏi đơn giản trước khi dùng pre-retrieval phức tạp
- [ ] Bật Hybrid retrieval nếu đã có sparse embedding
- [ ] Thiết lập top_k = 10-20
- [ ] Bật cross-encoder reranker cho production
- [ ] Bật redundancy filter
- [ ] Dùng sandwich ordering
- [ ] Chọn citation prompt template cho production
- [ ] Thiết lập temperature = 0.0
- [ ] Thêm instruction "không biết" vào system prompt
- [ ] Test với các câu hỏi nằm ngoài tài liệu (kiểm tra "I don't know")

Checklist Production

- [ ] Thiết lập input guardrails (PII, injection, length)
- [ ] Thiết lập output guardrails (faithfulness check)
- [ ] Implement query embedding cache
- [ ] Thiết lập monitoring (latency, faithfulness, token usage)
- [ ] Thiết lập alert khi metrics vượt ngưỡng
- [ ] Access control nếu có nhiều loại user
- [ ] Backup Vector DB định kỳ

- [ ] Kế hoạch incremental indexing khi corpus update

---

*Tài liệu này được viết cho project **RAG-pipeline-visualizer**. GitHub: https://github.com/vietnh1009/RAG-pipeline-visualizer Phiên bản: 1.0 | Ngôn ngữ: Tiếng Việt*